QUÉ PIDE, DÓNDE VERLO Y CÓMO EXPLICARLO

Cómo se cumple cada requerimiento

Los 16 requerimientos del proyecto y las dos funciones añadidas con el backend, uno por uno: qué pide cada uno, cómo mostrarlo en Folio, qué prueba lo demuestra y en qué parte del código está.



GUÍA DE REQUERIMIENTOS

Contenido

PARA EMPEZAR

Antes de empezar 3

Los requerimientos de un vistazo 6

Dónde aparece cada requerimiento 7

Parte I · Requerimientos funcionales 8

RF-01 Selección de nicho y temática cotidiana 9

RF-02 Temario de 5 a 7 capítulos 12

RF-03 Estructura estandarizada del capítulo 16

RF-04 Ejercicios prácticos rápidos 20

RF-05 Puntos de validación humana (HITL) 23

RF-06 Exportación a Markdown y PDF 27

Parte II · Requerimientos no funcionales 32

RNF-01 Tono y estilo de redacción 33

RNF-02 Menos de 60 segundos por capítulo 37

RNF-03 Consistencia y cohesión textual 41

RNF-04 Arquitectura modular de cuatro agentes 44

Parte III · Requerimientos de accesibilidad 48

RA-01 Jerarquía H1, H2 y H3 en Markdown y PDF 49

RA-02 Lenguaje claro y comprensible 53

RA-03 Contraste y tipografía limpia en el PDF 56

Parte IV · Requerimientos éticos y de responsabilidad 60

RÉR-01 Originalidad y prevención de plagio 61

RÉR-02 Aviso legal en temas sensibles 64

RÉR-03 Mitigación de sesgos y estereotipos 69

AÑADIDO EN LA FASE 4

Parte V · Funciones añadidas en la fase 4 72

RD-04 Edición de portada (ampliada) 73

RD-09 Ayuda con las referencias bibliográficas 79

PARA CERRAR

Guion de demostración en 5 minutos 84

Preguntas frecuentes 87

Lo que encontró la verificación 89

Qué trajo la fase 4 y qué falta 90

Glosario 92

Cómo repetir las pruebas 94

CÓMO ESTÁ HECHA

Las capturas las toma un recorrido automático por la app (`npm run guia:capturas`), que también mide dónde está cada elemento señalado. El PDF se arma con `npm run guia:pdf` a partir de esas capturas, los resultados de las pruebas y el código actual: los números de línea se buscan al generarlo.

La versión en texto, con las mismas evidencias, está en `docs/09-verificacion-requerimientos.md` .

CÓMO USAR ESTA GUÍA

Antes de empezar

Esta guía te acompaña para demostrar, uno por uno, los 16 requerimientos de `docs/requerimientos.txt` y las dos funciones que se añadieron con el backend (RD-04 y RD-09), y para explicarlos con tus palabras. Cada uno tiene su ficha: qué pide, qué decir, dónde tocar, qué prueba lo demuestra y en qué archivo está.

Dos formas de abrir Folio

Las dos tienen la misma interfaz y pasan por la misma fachada `api`; cambia quién hace el trabajo. Las capturas de esta guía son del modo demo.

Modo demo · sitio publicado

Los cuatro agentes se simulan en el navegador con textos de plantilla y los libros se guardan en tu navegador. Es lo que publica GitHub Pages: sirve para mostrar todo el recorrido sin instalar nada.

Con backend · en tu computador

FastAPI y LangGraph con los cuatro agentes reales, Gemini como modelo, datos en SQLite y el PDF generado en el servidor. En Windows, `.\iniciar-folio.ps1` desde la raíz lo arranca todo; crea una cuenta y escribe tu idea.

ABRE EL MODO DEMO EN TU TELÉFONO O COMPUTADOR

`franjmd0508.github.io/multiagente-inteligente-ebooks/`

1. Escanea el código o escribe la dirección.
2. Toca **“Continuar con Google”**. En el modo demo el acceso es simulado: no pide ninguna cuenta real.
3. Para ir rápido, abre **Ajustes → Demostración → Velocidad de los agentes: Rápida**. Para medir tiempos (RNF-02), déjala en Normal.
4. Si la biblioteca queda desordenada, usa **Ajustes → Demostración → Restaurar ejemplos**.

Los libros de ejemplo

En el modo demo, la biblioteca trae cuatro libros en distintas etapas. Te ahorran esperar cuando quieres mostrar una pantalla concreta.

Semanas que rinden
Gestión del tiempo

Tu turno · índice

RF-02 RF-05 RÉR-02

Hablar claro sin pelear Resolver conflictos	Tu turno · capítulo 1	RF-03 RF-05 RNF-01 RNF-03
Ahorra sin dejar de vivir Finanzas para universitarios	Listo · 6 capítulos	RF-04 RF-06 RA-01 RA-03 RÉR-01 RÉR-02 RÉR-03 RD-04 RD-09
Tu primer empleo sin miedo Primer empleo	Listo · 5 capítulos	RF-02 RÉR-02 (sin aviso)

Cómo leer cada ficha

En las capturas, lo que importa queda iluminado y el resto se oscurece. Los números verdes coinciden con la lista que hay debajo de cada captura.

1. **Qué pide el documento**
   El texto original de requerimientos.txt, palabra por palabra. En las funciones añadidas, el de docs/02.
2. **Tu turno · dilo así**
   La explicación corta para decir en voz alta. Marcada con la cinta, como en Folio.
3. **Cómo lo cumple**
   Lo que hace el sistema para cumplirlo, en el modo demo y con el backend.
4. **Dónde verlo, paso a paso**
   Qué tocar en el sitio y una captura por paso, con lo importante señalado.
5. **Cómo se comprueba**
   Las pruebas automáticas que lo demuestran y lo que midieron.
6. **Dónde está en el código**
   Archivo y línea, para abrirlo si te lo piden. Las rutas src/ y e2e/ son de frontend/.
7. **Qué falta medir**
   Solo aparece si queda algo por medir con el modelo real.
8. **Si te preguntan…**
   Preguntas probables del jurado, con su respuesta.

Qué significa cada estado

Cumple	Funciona en el sistema, en el modo demo y con el backend, y lo comprueba una prueba automática.
Implementado; falta medirlo con el modelo real	El backend ya tiene los mecanismos (prompts con reglas explícitas, capítulo ancla, guía de estilo, tiempo límite, verificación) y las pruebas pasan con el proveedor demo, pero la calidad del texto se tiene que medir con capítulos escritos por Gemini.

> **LA IDEA CLAVE PARA EXPLICAR LOS "IMPLEMENTADO; FALTA MEDIRLO CON EL MODELO REAL"**
>
> Los cuatro agentes ya existen en el backend. Los requerimientos sobre **cómo escribe la IA** (tono, tiempos, coherencia, claridad, originalidad y sesgos) tienen su mecanismo: reglas explícitas en los prompts, el capítulo 1 como ancla, una guía de estilo, un tiempo límite y verificaciones. Lo que falta es medir el resultado con capítulos escritos por Gemini: en la verificación no había clave, así que las pruebas usaron el proveedor demo, que devuelve textos de plantilla. Decirlo así es honesto y muestra que el equipo sabe qué falta.

RESUMEN

Los requerimientos de un vistazo

Los 16 requerimientos de `requerimientos.txt` y las 2 funciones añadidas en la fase 4: 12 se cumplen por completo y 6 están implementados, pero falta medirlos con el modelo real. Pasan las 20 pruebas del frontend, las 147 del backend y la de punta a punta.

12 Cumple · **6** Implementado; falta medirlo con el modelo real · **168 de 168** pruebas superadas

ID	REQUERIMIENTO	DÓNDE LO VES	ESTADO	PÁG.
RF-01	Selección de nicho y temática cotidiana	Inicio	Cumple	9
RF-02	Temario de 5 a 7 capítulos	Inicio y revisión del índice	Cumple	12
RF-03	Estructura estandarizada del capítulo	Revisión del capítulo 1 y “Tu libro”	Cumple	16
RF-04	Ejercicios prácticos rápidos	Final de cada capítulo	Cumple	20
RF-05	Puntos de validación humana (HITL)	Revisión del índice y del capítulo 1	Cumple	23
RF-06	Exportación a Markdown y PDF	Exportar	Cumple	27
RNF-01	Tono y estilo de redacción	Revisión del capítulo 1	Implementado; falta medirlo con el modelo real	33
RNF-02	Menos de 60 segundos por capítulo	Escribiendo	Implementado; falta medirlo con el modelo real	37
RNF-03	Consistencia y cohesión textual	Pausa 2 y redacción	Implementado; falta medirlo con el modelo real	41
RNF-04	Arquitectura modular de cuatro agentes	Backend, fases y bitácora	Cumple	44
RA-01	Jerarquía H1, H2 y H3 en Markdown y PDF	Exportar y editor	Cumple	49
RA-02	Lenguaje claro y comprensible	El texto del libro	Implementado; falta medirlo con el modelo real	53
RA-03	Contraste y tipografía limpia en el PDF	Portada y estilo, vista de impresión	Cumple	56
RÉR-01	Originalidad y prevención de plagio	Exportar y archivos exportados	Implementado; falta medirlo con el modelo real	61
RÉR-02	Aviso legal en temas sensibles	Índice, libro y editor	Cumple	64
RÉR-03	Mitigación de sesgos y estereotipos	Ejemplos cotidianos	Implementado; falta medirlo con el modelo real	69
RD-04	Edición de portada (ampliada)	Portada y estilo	Cumple	73
RD-09	Ayuda con las referencias bibliográficas	Pestaña Referencias y Exportar	Cumple	79

MAPA

Dónde aparece cada requerimiento

El recorrido de un libro en Folio tiene ocho paradas. En las dos marcadas con la cinta decides tú; en el resto trabaja un agente. Usa este mapa para saber en qué pantalla mostrar cada requerimiento.

1 **Inicio** — RF-01 RF-02 RÉR-02

Tú

Eliges un nicho o escribes tu idea. La ficción se rechaza con alternativas.

2 **Investigación** — RF-02 RNF-04

Agente Investigador

Propone título, subtítulo e índice de 5 a 7 capítulos.

3 **Revisión del índice** — RF-05 RF-02 RÉR-02 RD-09

Tú · primera pausa

Editas, reordenas, agregas o quitas capítulos y apruebas. Entonces el Investigador sugiere lecturas.

4 **Capítulo 1** — RF-03 RF-04 RNF-02

Redactor + Ejercicios

Se escribe en vivo, sección por sección, y termina con su ejercicio.

5 **Revisión del capítulo 1** — RF-05 RNF-01 RNF-03

Tú · segunda pausa

Ajustas el tono y apruebas. Este capítulo marca el resto.

6 **Capítulos 2 en adelante** — RNF-02 RNF-03 RA-02 RÉR-03

Redactor + Ejercicios

Se escriben uno detrás de otro, con ejemplos de personas diversas.

7 **Libro final** — RA-01 RA-03 RÉR-02 RD-04 RD-09

Agente Maquetador

Revisas el texto, la portada, la contraportada y las referencias, y eliges letra, tamaño y página.

8 **Exportar** — RF-06 RA-01 RÉR-01 RD-09

Tú

Descargas el libro en Markdown o PDF.

Folio

PARTE I

Requerimientos funcionales

Lo que Folio hace: las funciones que ves y tocas, desde elegir el tema hasta descargar el libro.

RF-01	Selección de nicho y temática cotidiana	9
RF-02	Temario de 5 a 7 capítulos	12
RF-03	Estructura estandarizada del capítulo	16
RF-04	Ejercicios prácticos rápidos	20
RF-05	Puntos de validación humana (HITL)	23
RF-06	Exportación a Markdown y PDF	27

REQUERIMIENTO FUNCIONAL · 1 DE 6

Cumple

RF-01

Selección de nicho y temática cotidiana

QUÉ PIDE EL DOCUMENTO

El sistema debe permitir seleccionar o parametrizar nichos de crecimiento personal basados en situaciones reales (ej. finanzas para universitarios, gestión del tiempo, resolución de conflictos) descartando explícitamente contenido de ficción.

TU TURNO · DILO ASÍ

En el inicio eliges un nicho de la vida real o escribes tu propia idea. Si la idea es de ficción, Folio no crea el libro y te propone tres alternativas prácticas. Con el backend, el servidor también lo comprueba.

Cómo lo cumple

- El inicio ofrece cinco nichos como chips: Finanzas para universitarios, Gestión del tiempo, Resolver conflictos, Primer empleo y Hábitos y rutinas.
- Un chip llena la caja con una idea editable. Si escribes tu propia idea, Folio detecta su nicho por las palabras clave (el catálogo tiene ocho).
- Si la idea suena a ficción (novela, cuento, poema, dragones…), no se crea el ebook: aparece “Folio crea guías prácticas sobre situaciones reales” con tres alternativas.
- Con el backend, el inicio consulta al servidor antes de crear el libro: patrones de ficción más una clasificación del modelo de lenguaje. El grafo lo vuelve a validar antes de que trabaje el Investigador (D-41).

Cómo se comprueba

`RF-01 · Nichos de situaciones reales; la ficción se rechaza`

Que hay cinco nichos o más, que un chip rellena la idea y que una idea de ficción no crea ningún ebook. En el backend, `test_la_ficcion_se_rechaza_con_alternativas` hace lo mismo por la API.

Prueba superada · **5** nichos en el inicio · **0** ebooks creados con ficción

Dónde verlo, paso a paso

1 Abre el inicio y toca un chip

Entra con “Continuar con Google”. En el inicio verás los nichos como chips; toca “Gestión del tiempo”.



1. Los nichos de situaciones reales, como chips. Toca uno…
2. …y la caja se llena con una idea que puedes editar.

2 Pide un libro de ficción

Borra la caja, escribe “Escribe una novela de dragones para adolescentes” y toca la flecha. Después abre la Biblioteca: no se creó ningún libro.



1. Escribes una idea de ficción y la envías.
2. El libro no se crea: Folio explica que escribe guías prácticas sobre situaciones reales…
3. …y propone tres alternativas de no ficción. Tocar una la pone en la caja.

Dónde está en el código

`src/lib/niches.ts:16`	Catálogo de nichos, con su sensibilidad y su color de tela
`src/lib/niches.ts:115`	Detecta el nicho de una idea escrita a mano
`src/lib/guardrails.ts:23`	Guardarraíl de ficción (lista de patrones)
`src/app/(app)/page.tsx:55`	Consulta el tema antes de crear el libro y muestra las alternativas
`src/lib/api/mock/api.ts:34`	Modo demo: aplica los patrones en el navegador
`backend/app/graph/guardrail.py:121`	Con backend: patrones y clasificación del modelo
`backend/app/graph/nodes.py:113`	Segunda validación, dentro del grafo

Si te preguntan…

¿Cómo sabe que algo es ficción?
En el modo demo, con una lista de patrones (novela, cuento, poema, saga, personajes, dragones…). Con el backend, además, el modelo de lenguaje clasifica la idea antes de crear el libro, y el grafo lo vuelve a comprobar antes de que arranque el Investigador. La respuesta amable es la misma.

¿Y si escribo un tema que no está en los chips?
Se acepta si es de no ficción. Folio lo asigna al nicho más parecido o, si no encaja en ninguno, a “Crecimiento personal”, que lleva aviso legal de autoayuda.

¿Por qué no mostrar un simple error?
Porque el tono de Folio es empático (RNF-01): en lugar de un “no”, ofrece tres caminos parecidos que sí puede escribir.

REQUERIMIENTO FUNCIONAL · 2 DE 6

Cumple

RF-02

Temario de 5 a 7 capítulos

QUÉ PIDE EL DOCUMENTO

El Agente Investigador debe generar una propuesta de estructura con un límite estricto de entre 5 y 7 capítulos.

TU TURNO · DILO ASÍ

El índice siempre tiene entre 5 y 7 capítulos. Con 5 no puedes quitar y con 7 no puedes agregar. Y aunque alguien se saltara la interfaz, la capa de datos y el backend vuelven a validar el rango.

Cómo lo cumple

- Al crear el ebook solo hay cuatro opciones: 5, 6, 7 o automático (en el modo demo, el Investigador propone 6).
- En la revisión del índice, con 5 capítulos las papeleras se desactivan; con 7, el botón dice “7 capítulos · máximo alcanzado” y no responde.
- La regla también vive en la capa de datos (`addOutlineItem` y `removeOutlineItem`, en `local.ts`), no solo en los botones.
- Con el backend, el Investigador recibe el rango en su prompt, el código descarta un índice fuera de rango y la API rechaza cualquier cambio que deje menos de 5 o más de 7 capítulos.

Cómo se comprueba

`RF-02 · El temario siempre tiene entre 5 y 7 capítulos`

Crea un libro de 5 capítulos (quitar desactivado), sube a 7 (agregar desactivado) y comprueba que “automático” da 6. En el backend, `test_nicho_general_se_ajusta_y_validaciones_del_indice` intenta guardar índices fuera de rango.

Prueba superada · **5** → quitar desactivado · **7** → agregar desactivado · Automático = **6**

Dónde verlo, paso a paso

1 Abre las opciones de capítulos

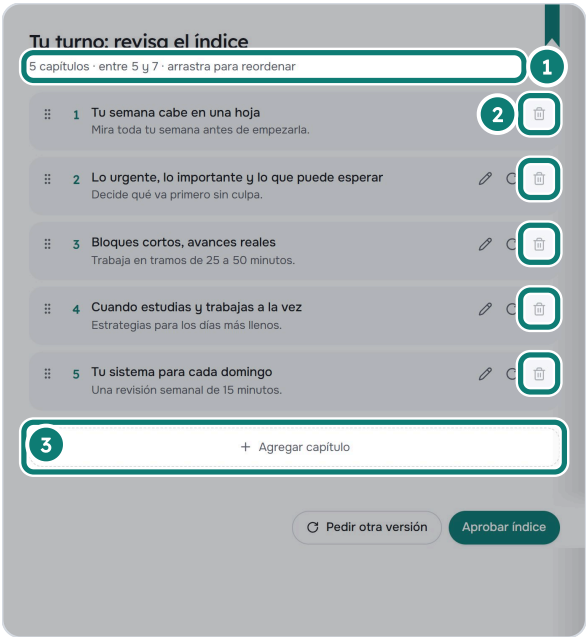
En el inicio, toca la píldora “Capítulos: automático”, debajo de la caja.



1. La píldora “Capítulos” abre las opciones…
2. …y solo hay cuatro: 5, 6, 7 o automático. No se puede pedir otro número.

2 Crea el libro con 5 capítulos

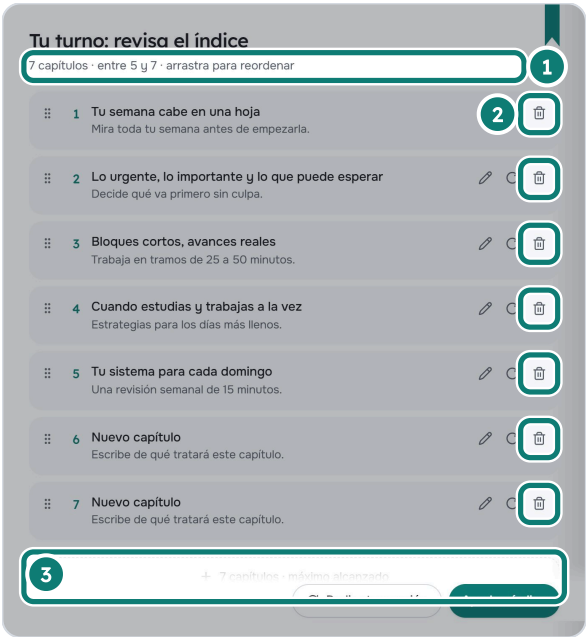
Elige 5, toca un chip y envía. Cuando aparezca “Tu turno: revisa el índice”, fíjate en los botones de cada capítulo.



1. El rango siempre a la vista: 5 capítulos, entre 5 y 7.
2. Con 5, las papeleras están desactivadas: no se puede bajar del mínimo.
3. “Agregar capítulo” sigue disponible hasta llegar a 7.

3 Intenta pasar de 7

Toca “Agregar capítulo” dos veces.



1. Ahora son 7 capítulos, el máximo.
2. Las papeleras vuelven a funcionar: se puede quitar.
3. El botón de agregar se desactiva y lo explica: “7 capítulos · máximo alcanzado”.

Dónde está en el código

`src/lib/types.ts:244`	Límites: MIN_CHAPTERS = 5 y MAX_CHAPTERS = 7
`src/lib/api/local.ts:57`	No deja pasar de 7 (los dos modos)
`src/lib/api/local.ts:77`	No deja bajar de 5 (los dos modos)
`src/components/workspace/stages/OutlineReview.tsx:186`	Botones y mensaje del límite
`src/lib/api/mock/content.ts:308`	Capítulos que propone el Investigador (modo demo)
`backend/app/agents/investigador.py:78`	Con backend: valida el índice del Investigador
`backend/app/api/ebooks.py:300`	La API rechaza otro número

Si te preguntan…

¿Qué pasa si pido otra versión del índice?
El Investigador propone otro temario, siempre dentro del rango. También puedes regenerar un solo capítulo con el ícono de flecha circular.

¿Por qué validar dos veces?
Porque la interfaz se puede saltar, por ejemplo llamando a la API directamente. Una regla de negocio tiene que vivir donde están los datos, no solo en los botones.

¿“Automático” siempre es 6?
En el modo demo, sí. Con el backend, el Investigador elige 5, 6 o 7 según el tema; si propone más, el código recorta el índice y, si propone menos, lo descarta y lo vuelve a pedir.

REQUERIMIENTO FUNCIONAL · 3 DE 6

Cumple

RF-03

Estructura estandarizada del capítulo

QUÉ PIDE EL DOCUMENTO

Cada capítulo generado debe incluir de manera obligatoria: Introducción, Desarrollo (con tono conversacional y directo), Ejemplos cotidianos, Conclusión y Ejercicio práctico de cierre.

TU TURNO · DILO ASÍ

Todos los capítulos traen las mismas cinco secciones, en el mismo orden. Puedes editar lo que dicen, pero no puedes borrarlas.

Cómo lo cumple

- El orden está fijado en el código (`SECTION_ORDER`): Introducción, Desarrollo, Ejemplos cotidianos y Conclusión. El Agente de Ejercicios agrega la quinta, Ejercicio práctico.
- En la revisión del capítulo 1, la lista “Estructura” marca las cinco con un visto.
- En la revisión final, cada sección tiene su nombre fijo con un candado: cambias el texto, no la estructura.
- Con el backend, el prompt del Redactor exige exactamente esas cuatro secciones, en ese orden. El código comprueba el capítulo al terminar y, si falta alguna, lo vuelve a pedir.

Cómo se comprueba

`RF-03 · Cada capítulo tiene las cinco secciones obligatorias en orden`

Lee los encabezados de sección de cada capítulo y exige exactamente esos cinco, en ese orden. En el backend, `test_flujo_completo` comprueba las cuatro secciones y el ejercicio del capítulo 1.

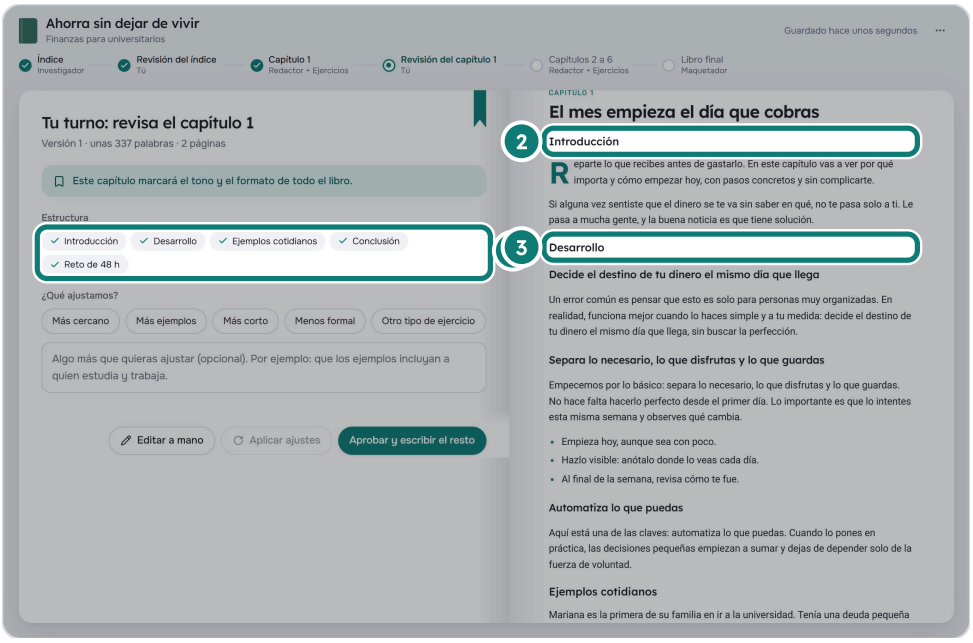
Prueba superada

6 de 6 capítulos con las 5 secciones

Dónde verlo, paso a paso

1 Llega a la revisión del capítulo 1

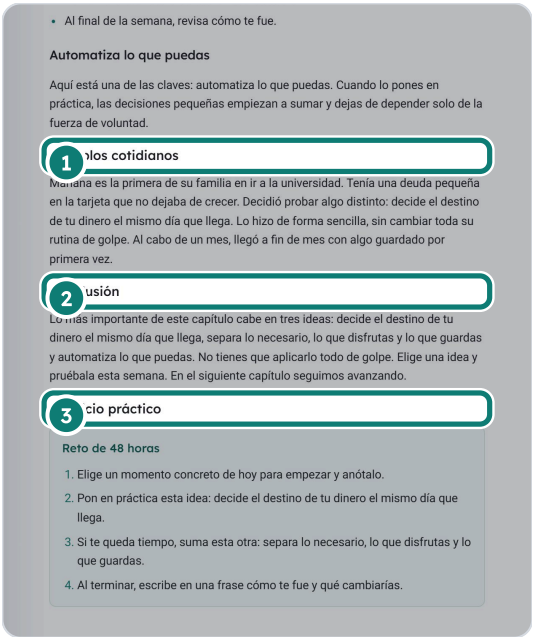
Aprueba el índice de un libro nuevo, o abre “Hablar claro sin pelear”, que ya está en esa pausa.



1. La lista “Estructura” marca las cinco secciones del capítulo, en orden.
2. En “Tu libro”, el capítulo empieza por la Introducción…
3. …y sigue con el Desarrollo.

2 Baja hasta el final del capítulo

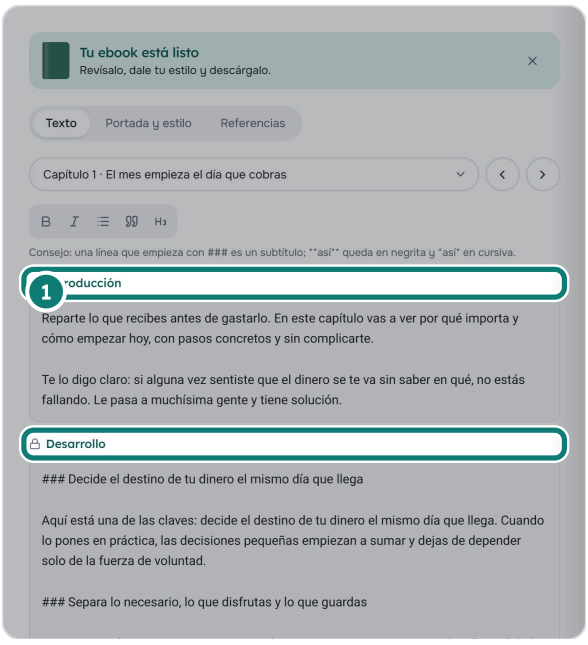
Desplázate por “Tu libro”, a la derecha.



1. Después vienen los Ejemplos cotidianos…
2. …la Conclusión…
3. …y el Ejercicio práctico, que prepara el Agente de Ejercicios.

3 Intenta cambiar la estructura

En un libro listo, abre la pestaña “Texto”.



1. En “Texto”, cada sección tiene su nombre fijo con candado: cambias el contenido, no la estructura.

Dónde está en el código

`src/lib/types.ts:25`	Orden fijo de las secciones
`src/lib/api/mock/content.ts:362`	El Redactor simulado genera las cuatro primeras
`src/components/book/ChapterPage.tsx:40`	Hoja del capítulo, en ese orden
`src/components/book/ChapterPage.tsx:84`	Lista “Estructura” de la pausa 2
`src/components/workspace/stages/TextEditor.tsx:200`	Editor con secciones fijas
`backend/app/prompts/redactor/capitulo.md:16`	Con backend: el prompt fija las cuatro secciones
`backend/app/graph/nodes.py:473`	Si falta una sección, el capítulo se vuelve a pedir

Si te preguntan…

¿Y el tono conversacional del Desarrollo?
Se explica en RNF-01. Lo que exige este requerimiento es que cada sección exista y esté en su lugar.

¿Puedo agregar una sección extra?
Puedes agregar subtítulos (`###`) dentro de una sección, pero no secciones nuevas ni quitar las que hay.

¿Por qué el ejercicio lo hace otro agente?
Porque RNF-04 pide separar funciones: el Redactor escribe las cuatro primeras secciones y el Agente de Ejercicios diseña el cierre con las reglas de RF-04. Cada uno tiene su módulo y su prompt.

REQUERIMIENTO FUNCIONAL · 4 DE 6

Cumple

RF-04

Ejercicios prácticos rápidos

QUÉ PIDE EL DOCUMENTO

El sistema debe estar programado para generar únicamente dinámicas de acción rápida como checklists de autoevaluación o retos de 24 a 48 horas.

TU TURNO · DILO ASÍ

El ejercicio solo puede ser de dos tipos: un checklist de autoevaluación o un reto de 24 o 48 horas. En el modelo de datos no existe una tercera opción.

Cómo lo cumple

- El tipo `Exercise` es cerrado: `checklist`, o `reto` con duración `24h` o `48h`. Cualquier otro valor no compila.
- Por defecto alternan: retos en los capítulos impares y checklists en los pares. En Ajustes → Ejercicio preferido puedes fijar uno, y en la pausa 2 está “Otro tipo de ejercicio”.
- En el libro aparece como un recuadro al final del capítulo: un reto con pasos numerados o una lista para marcar.
- Con el backend, el orquestador decide el tipo con la misma regla, el prompt del Agente de Ejercicios la repite y el código valida la respuesta: de 5 a 7 afirmaciones en un checklist, o de 3 a 5 pasos y 24 o 48 horas en un reto.

Cómo se comprueba

`RF-04 · Solo checklists de autoevaluación o retos de 24-48 horas`

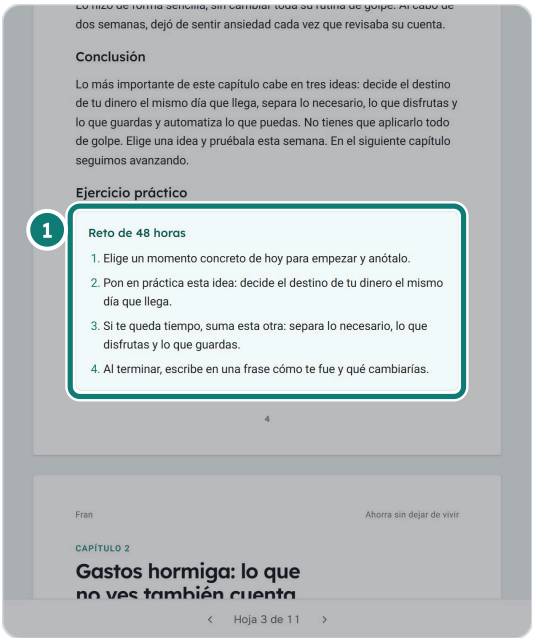
Revisa el ejercicio de los seis capítulos del libro de prueba y exige que cada uno sea un checklist o un reto de 24 o 48 horas. En el backend, `test_validacion_de_ejercicios_rf04` prueba la validación.

Prueba superada · Cap. 1 · **Reto 48 h** · Cap. 2 · **Checklist** · Cap. 3 · **Reto 24 h** · Cap. 4 · **Checklist** · Cap. 5 · **Reto 48 h** · Cap. 6 · **Checklist**

Dónde verlo, paso a paso

1 Mira el final del capítulo 1

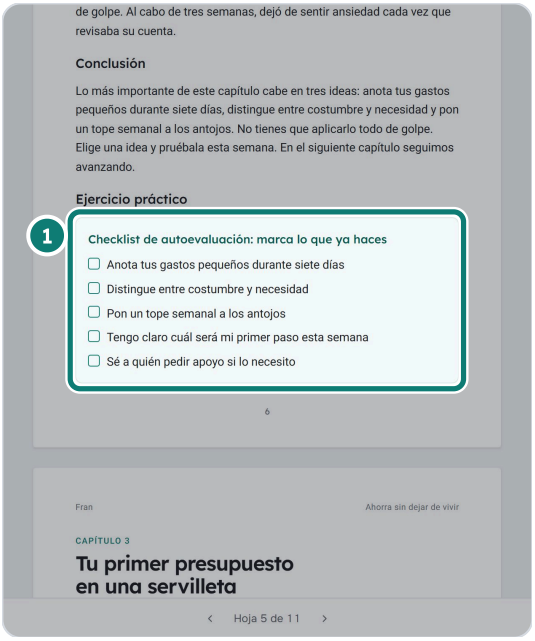
Abre “Ahorra sin dejar de vivir” (listo) y baja en “Tu libro” hasta “Ejercicio práctico”.



1 Final del capítulo 1: “Reto de 48 horas”, con pasos concretos.

2 Compáralo con el capítulo 2

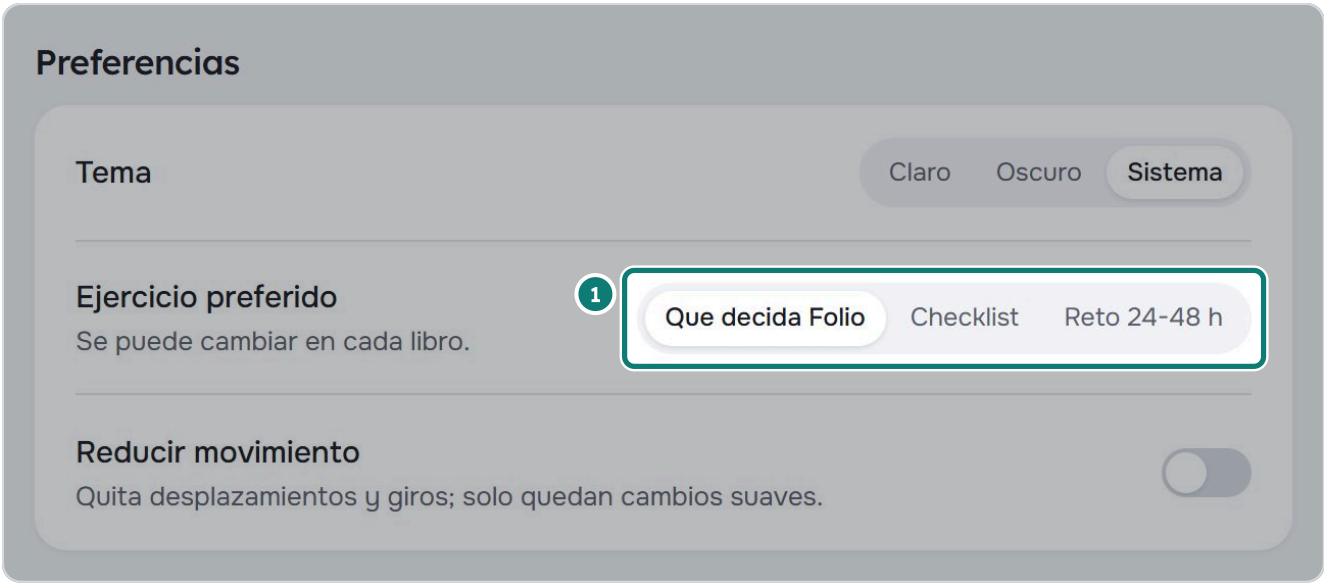
Sigue bajando hasta el ejercicio del capítulo 2.



1 Final del capítulo 2: “Checklist de autoevaluación: marca lo que ya haces”, una lista para marcar.

3 Muestra la preferencia

En Ajustes → Preferencias está “Ejercicio preferido”. Solo ofrece las dos formas permitidas.



1 “Ejercicio preferido”: que decida Folio (alterna), solo checklist o solo reto de 24-48 h.

Dónde está en el código

`src/lib/types.ts:36`	Tipo cerrado: checklist, o reto de 24 h o 48 h
`src/lib/api/mock/content.ts:446`	Agente de Ejercicios (modo demo)
`src/components/book/ChapterPage.tsx:7`	Recuadro del ejercicio en el libro
`backend/app/agents/ejercicios.py:19`	Con backend: el tipo según la preferencia
`backend/app/agents/ejercicios.py:82`	Valida el tipo, la duración y la extensión

Si te preguntan…

¿Por qué alternan?

Para dar ritmo al libro: un capítulo te hace actuar y el siguiente te hace revisar. Si prefieres uno solo, lo eliges en Ajustes.

¿Y si alguien quiere un ejercicio de una semana?

No se puede: la duración solo admite 24 o 48 horas. Con el backend, el prompt del Agente de Ejercicios repite la regla y el código revisa su respuesta antes de guardarla.

REQUERIMIENTO FUNCIONAL · 5 DE 6

Cumple

RF-05

Puntos de validación humana (HITL)

QUÉ PIDE EL DOCUMENTO

El sistema debe detener su ejecución autónoma en dos fases críticas para requerir la aprobación del usuario: (1) pausa tras la propuesta de índice y temas de los capítulos; (2) pausa tras la redacción final del primer capítulo para validar tono y formato antes de proceder con el resto.

TU TURNO · DILO ASÍ

Folio se detiene dos veces y no sigue sin ti: después del índice y después del capítulo 1. La pausa se guarda, así que aunque cierres el navegador, al volver sigue esperando.

Cómo lo cumple

- Al terminar el índice, el libro queda en el estado `indice_pendiente` y solo “Aprobar índice” lo mueve. Lo mismo pasa con `cap1_pendiente` y “Aprobar y escribir el resto”.
- La pausa se nota: cae la cinta marcapáginas, la fase activa dice “Tú” y en la biblioteca el libro aparece como “Tu turno”.
- En cada pausa puedes editar, reordenar, pedir otra versión o ajustar el tono antes de aprobar.
- El estado se guarda: al recargar, el sistema retoma el libro en el mismo punto.
- Con el backend, el grafo de LangGraph se detiene con `interrupt()` y guarda su estado en SQLite: aunque se reinicie el servidor, el libro sigue en la misma pausa.

Cómo se comprueba

`RF-05 · Pausa 1: el sistema se detiene tras proponer el índice`

`RF-05 · Pausa 2: se detiene tras escribir el capítulo 1`

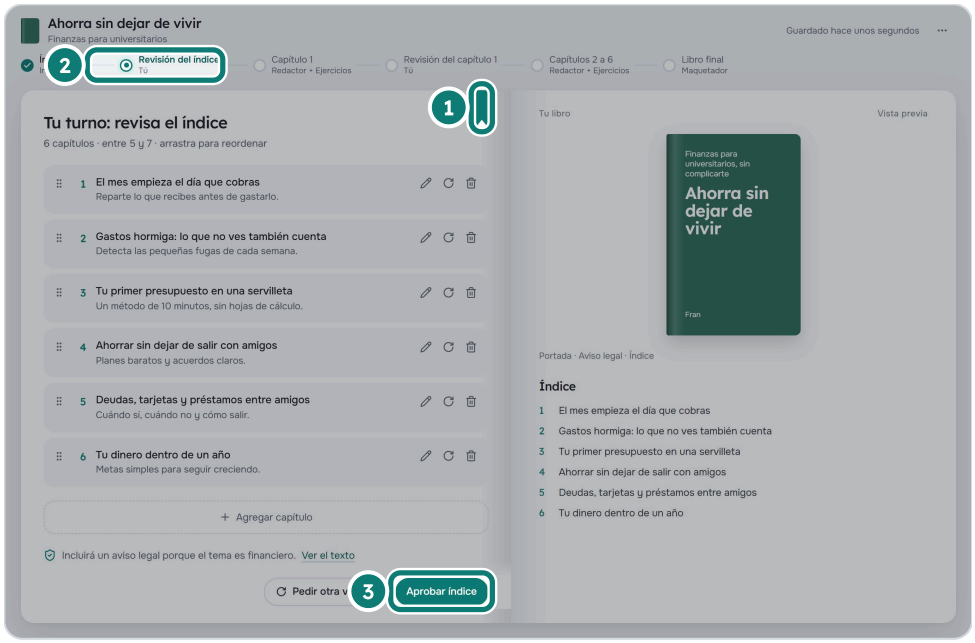
Espera con el libro en pausa y comprueba que no se escribió ningún capítulo; recarga y verifica que sigue en pausa; y confirma que los capítulos 2 en adelante esperan la segunda aprobación. En el backend, `test_retomar_una_pausa_tras_reiniciar` reinicia el servidor en plena pausa, y la prueba punta a punta recarga el navegador en la primera.

Prueba superada | **0** capítulos escritos durante la pausa | La pausa **sobrevive** a recargar

Dónde verlo, paso a paso

1 Crea un libro y espera la primera pausa

Cuando el Investigador termina el índice, el sistema se detiene. Espera todo lo que quieras o recarga la página: sigue en el mismo punto.



1. Cae la cinta marcapáginas: es tu turno.
2. La fase activa es “Revisión del índice · Tú”.
3. Mientras no toques “Aprobar índice”, no se escribe nada.

2 Mira la Biblioteca

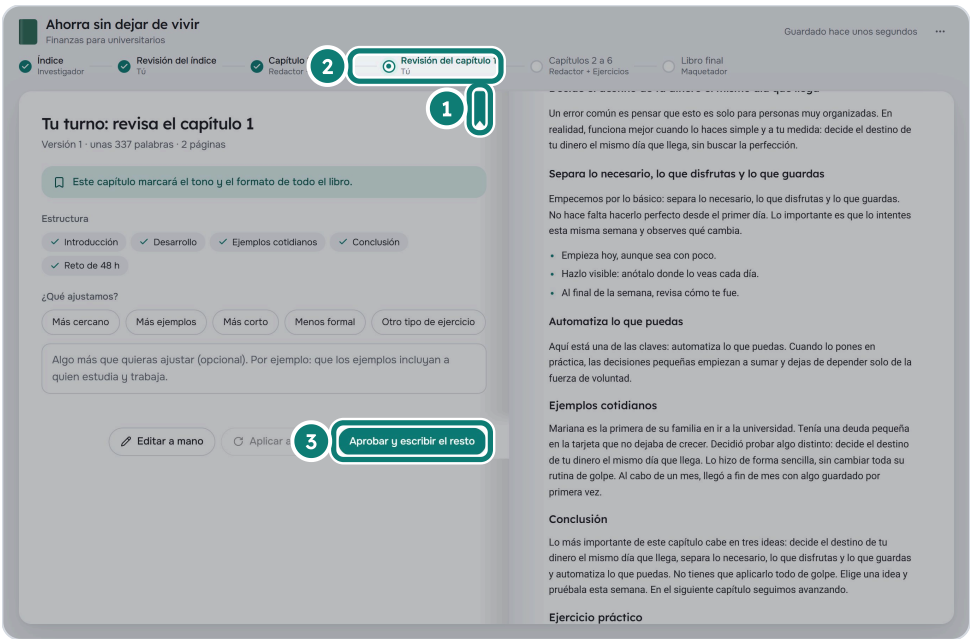
Sal del libro y abre la Biblioteca.



1. El libro que espera tu aprobación lleva la cinta en la portada…
2. …y el filtro “Tu turno” reúne todos los libros en pausa.

3 Aprueba el índice y llega a la segunda pausa

Vuelve al libro y toca “Aprobar índice”. El Redactor escribe solo el capítulo 1 y se detiene otra vez.



1. Otra vez la cinta: segunda pausa.
2. La fase activa es “Revisión del capítulo 1 · Tú”; los capítulos 2 en adelante esperan.
3. Solo al tocar “Aprobar y escribir el resto” se escribe lo demás.

Dónde está en el código

`src/lib/api/mock/simulator.ts:125`	Modo demo · pausa 1: termina en indice_pendiente
`src/lib/api/mock/simulator.ts:200`	Modo demo · pausa 2: termina en cap1_pendiente
`src/lib/api/mock/simulator.ts:335`	Modo demo: retoma el trabajo tras recargar
`src/lib/phases.ts:33`	Fases con “Tú” en las pausas
`backend/app/graph/nodes.py:352`	Con backend · pausa 1: interrupt() tras el índice
`backend/app/graph/nodes.py:584`	Con backend · pausa 2: interrupt() tras el capítulo 1
`backend/app/graph/runner.py:33`	Reanuda el grafo al aprobar y tras un reinicio

Si te preguntan…

¿Qué es HITL?

Human in the loop: una persona dentro del circuito. El sistema automatiza el trabajo, pero en los puntos críticos decide un humano.

¿Cómo funciona con el backend?

LangGraph tiene la función `interrupt()`, que detiene el grafo de agentes y guarda su estado en un *checkpointer* persistente, aquí en SQLite. Al aprobar, la API reanuda el grafo justo en ese punto. Si el servidor se reinicia, cada libro sigue donde estaba.

¿Por qué justo esas dos pausas?

El índice define de qué trata el libro y el capítulo 1 define cómo suena. Corregir ahí es barato; corregir con seis capítulos escritos, no.

REQUERIMIENTO FUNCIONAL · 6 DE 6

Cumple

RF-06

Exportación a Markdown y PDF

QUÉ PIDE EL DOCUMENTO

El sistema debe entregar el producto final formateado en Markdown (.md) con sintaxis limpia (encabezados, listas, bloques de citas) y permitir la compilación directa a un documento PDF estructurado y listo para maquetación o venta.

TU TURNO · DILO ASÍ

Cuando el libro está listo, Exportar te da dos formatos: Markdown limpio, para otras herramientas, y un PDF estructurado con la portada, la letra y el tamaño de página que elegiste.

Cómo lo cumple

- Markdown: portada (título, subtítulo y autor), aviso legal como cita (`>`), índice numerado, capítulos con encabezados, listas, checklists (`- [ ]`) y la nota de transparencia al final.
- PDF: se abre una vista de impresión con el tamaño elegido (A5, A4, Carta o 6 × 9 in) y se guarda como PDF desde el navegador. Sale etiquetado y con marcadores.
- Con el backend, los dos archivos los genera el Agente Maquetador en el servidor. El PDF sale de Chromium, etiquetado, con marcadores, las fuentes incrustadas y la marca de IA en los metadatos (D-36 y D-39).
- En el repositorio hay ejemplos reales en `docs/verificacion/`: `ebook-ejemplo.md` y `ebook-ejemplo.pdf` (24 páginas, modo demo), y `ebook-ejemplo-servidor.pdf` (24 páginas, generado por el backend).

Cómo se comprueba

`RF-06 · Exportación a Markdown y a PDF`

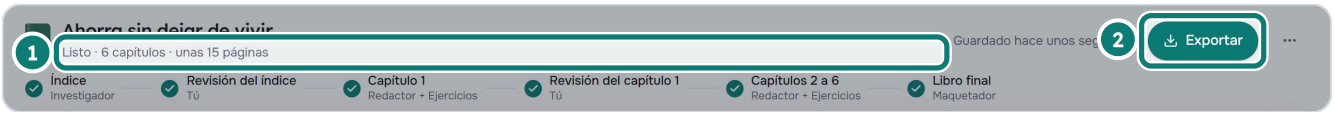
Descarga el Markdown y comprueba que tiene encabezados, listas y citas; después genera el PDF desde la vista de impresión. Con el backend, `test_flujo_completo` y la prueba punta a punta descargan los dos archivos del servidor.

Prueba superada · **59** encabezados · **51** elementos de lista · **15** casillas de checklist · **1** cita (aviso legal) · PDF de **24** páginas

Dónde verlo, paso a paso

1 Abre un libro listo

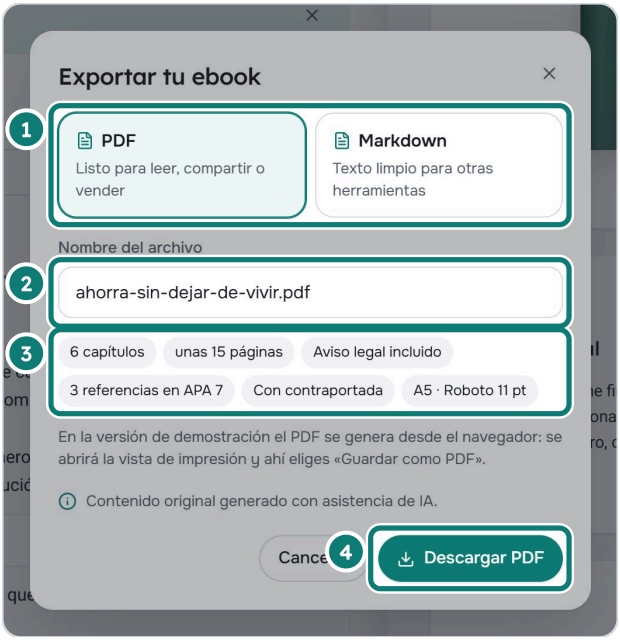
Abre “Ahorra sin dejar de vivir” desde la Biblioteca.



1. Cuando el libro está listo, lo dice aquí…
2. …y aparece el botón “Exportar”.

2 Toca Exportar

Se abre el diálogo de exportación.



1. Eliges el formato: PDF o Markdown.
2. El nombre del archivo, que puedes cambiar.
3. Un resumen: capítulos, páginas, aviso legal, tamaño de página y letra.
4. Y descargas.

3 Descarga el Markdown y ábrelo

Elige Markdown → “Descargar Markdown” y abre el archivo con cualquier editor de texto. Así empieza, y así se ve un ejercicio:

ahorra-sin-dejar-de-vivir.md

```
# Ahorra sin dejar de vivir                                                    (1)

*Finanzas para universitarios, sin complicarte*

Por Fran

## Aviso legal                                                                 (2)

> Este libro tiene fines educativos e informativos. No sustituye la asesoría de un profesional en finanzas. Antes de tomar decisiones importantes sobre tu dinero, consulta a una persona experta.   (3)

## Índice

1. El mes empieza el día que cobras                                           (4)
2. Gastos hormiga: lo que no ves también cuenta
3. Tu primer presupuesto en una servilleta
4. Ahorrar sin dejar de salir con amigos
5. Deudas, tarjetas y préstamos entre amigos
6. Tu dinero dentro de un año
```

1. El título del libro es un encabezado H1 (`#`).
2. Las partes del libro son H2 (`##`).
3. El aviso legal va como bloque de cita (`>`).
4. El índice es una lista numerada.

ahorra-sin-dejar-de-vivir.md

```
## Ejercicio práctico                                                          (1)

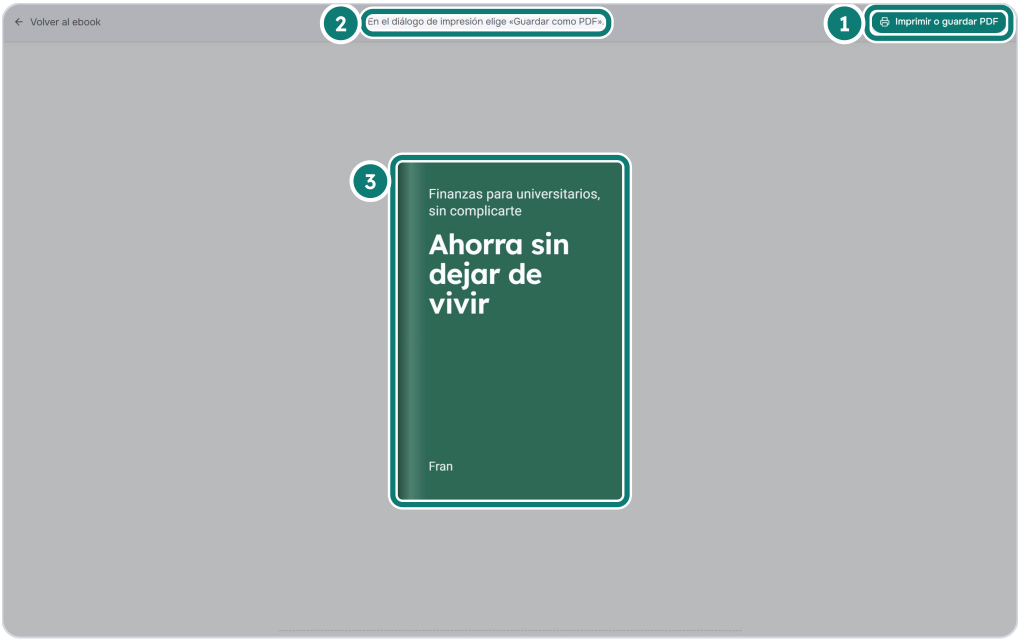
**Checklist de autoevaluación: marca lo que ya haces**

- [ ] Anota tus gastos pequeños durante siete días                            (2)
- [ ] Distingue entre costumbre y necesidad
- [ ] Pon un tope semanal a los antojos
```

1. Cada sección del capítulo es un H2.
2. Los checklists usan casillas (`- [ ]`).

4 Descarga el PDF

Vuelve a Exportar y elige PDF → “Descargar PDF”. En el modo demo se abre la vista de impresión; con el backend, el archivo se descarga directamente del servidor.



1. Tocas “Imprimir o guardar PDF”…
2. …y en el diálogo del navegador eliges “Guardar como PDF”.
3. El libro ya viene maquetado con el tamaño de página elegido, empezando por la portada.

Dónde está en el código

`src/lib/markdown.ts:17`	Arma el Markdown completo
`src/components/workspace/ExportDialog.tsx:42`	Diálogo de exportación
`src/app/imprimir/page.tsx:103`	Modo demo: vista de impresión con el tamaño de página
`backend/app/export/markdown.py:80`	Con backend: el Markdown en el servidor
`backend/app/export/pdf.py:106`	Con backend: PDF etiquetado con Chromium
`backend/app/export/pdf.py:195`	Título, autor y marca de IA en los metadatos

Si te preguntan…

¿Quién genera el PDF?

Depende del modo. En el sitio publicado (modo demo) no hay servidor, así que lo genera el motor de impresión de Chrome o Edge, que ya produce un PDF etiquetado. Con el backend, lo genera el Agente Maquetador en el servidor con Chromium sin ventana, con la misma portada y las mismas reglas de letra y contraste (D-36).

¿Qué lo hace “listo para maquetación o venta”?
Trae portada, aviso legal, índice y capítulos con jerarquía; usa tamaños de página estándar (6 × 9 in es habitual en autopublicación) y tipografía accesible. El Markdown sirve para llevarlo a otra herramienta de maquetación.

PARTE II

Requerimientos no funcionales

Cómo lo hace: el tono del texto, la velocidad, la coherencia entre capítulos y la arquitectura de los agentes.

RNF-01	Tono y estilo de redacción	33
RNF-02	Menos de 60 segundos por capítulo	37
RNF-03	Consistencia y cohesión textual	41
RNF-04	Arquitectura modular de cuatro agentes	44

REQUERIMIENTO NO FUNCIONAL · 1 DE 4

RNF-01

Tono y estilo de redacción

Implementado; falta medirlo con el modelo real

QUÉ PIDE EL DOCUMENTO

La IA debe mantener una temperatura/configuración de generación que garantice un tono conversacional, empático, directo e intermedio en profundidad (evitando explicaciones académicas densas o textos superficiales de rellenado).

TU TURNO · DILO ASÍ

El texto te habla de tú a tú, y en la revisión del capítulo 1 puedes ajustar el tono con un toque. Con el backend, el prompt del Redactor fija ese tono y usa temperatura 0,8. Falta leer capítulos escritos por Gemini para medirlo.

Cómo lo cumple

- El contenido del modo demo está escrito en segunda persona, con frases cortas y pasos concretos (“En este capítulo vas a ver…”, “Elige una idea y pruébala esta semana”).
- En la revisión del capítulo 1 hay cinco ajustes de un toque (Más cercano, Más ejemplos, Más corto, Menos formal y Otro tipo de ejercicio) y un campo libre.
- Cada ajuste crea una versión nueva (versión 2, 3…) y siempre puedes volver a la anterior.
- Con el backend, el prompt del Redactor pide tuteo, un tono conversacional, empático y directo y profundidad intermedia, y prohíbe una lista de muletillas de IA. Los ajustes del capítulo 1 llegan al Redactor, que lo reescribe.

Cómo se comprueba

`RNF-01 · Tono conversacional y ajustes de tono`

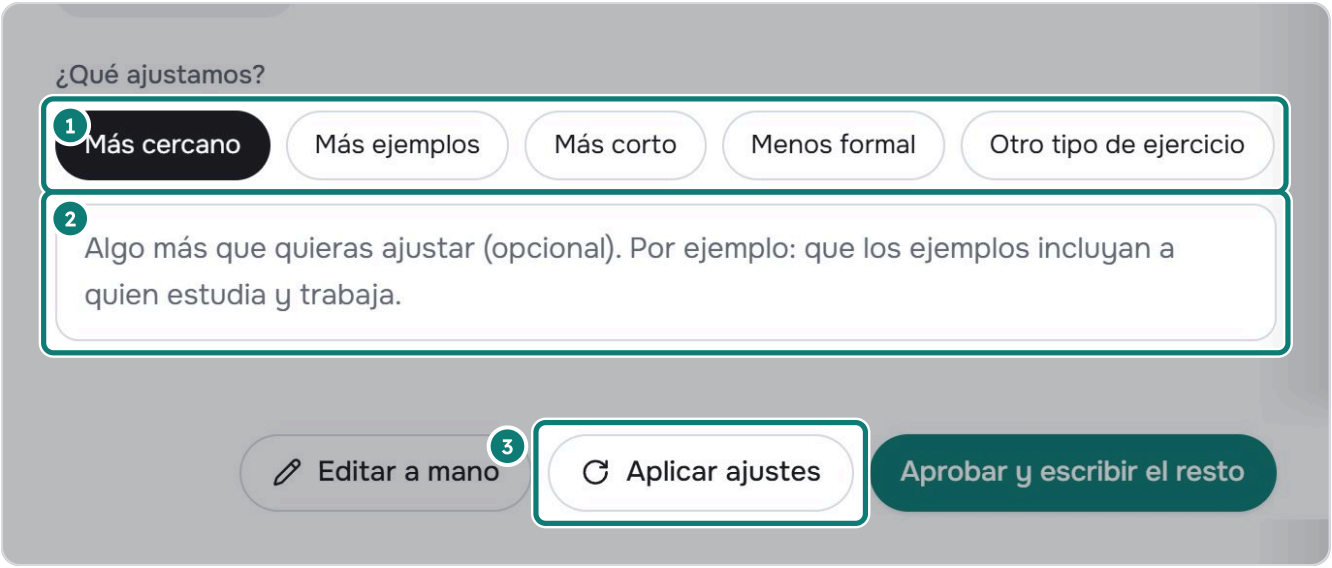
Comprueba el tuteo del texto, que están los cinco ajustes y que la versión 2 aplica el ajuste pedido. En el backend, `test_prompts_tienen_reglas_explicitas` comprueba que las reglas de tono siguen en los prompts.

Prueba superada · **5** ajustes de tono · Versión **2** con el ajuste aplicado

Dónde verlo, paso a paso

1 **Elige un ajuste en la revisión del capítulo 1**

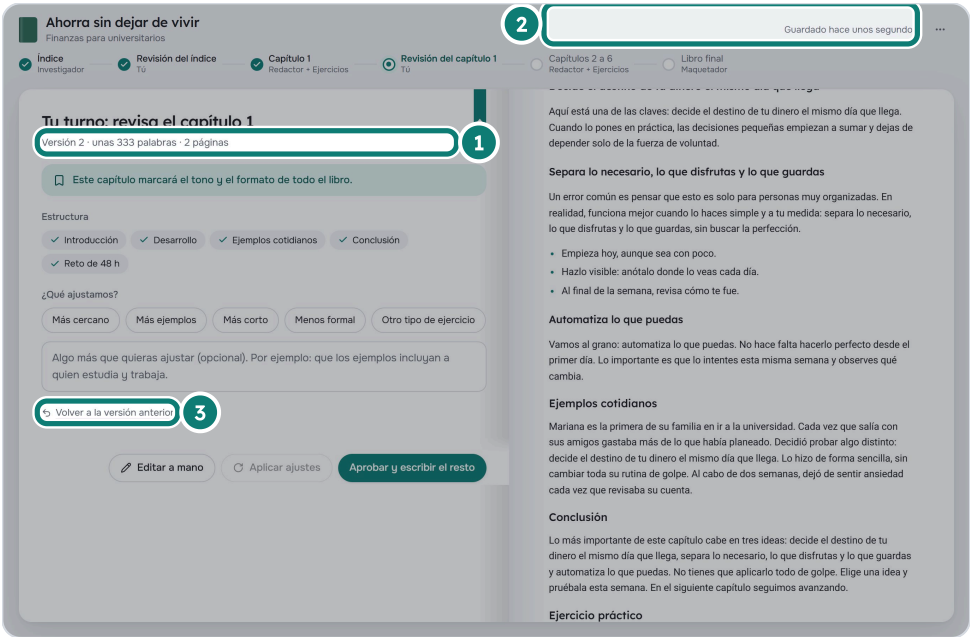
En la segunda pausa, toca “Más cercano”.



1. Cinco ajustes de un toque; aquí está elegido “Más cercano”.
2. Un campo libre para pedir otro cambio con tus palabras.
3. “Aplicar ajustes” genera una versión nueva del capítulo.

2 Aplica y compara

Toca “Aplicar ajustes” y espera unos segundos.



1. Ahora es la versión 2.
2. El texto cambió: la introducción empieza con “Te lo digo claro”, más cercano.
3. Siempre puedes volver a la versión anterior.

Dónde está en el código

`src/lib/types.ts:236`	Los cinco ajustes de tono
`src/lib/api/mock/content.ts:378`	Modo demo: efecto de “Más cercano” en el texto
`src/lib/api/mock/simulator.ts:309`	Modo demo: crea la nueva versión del capítulo 1
`backend/app/prompts/redactor/capitulo.md:3`	Con backend: temperatura del Redactor
`backend/app/prompts/redactor/capitulo.md:9`	Reglas de tono y muletillas prohibidas

Qué falta medir

Evaluar el tono de capítulos escritos por Gemini con la rúbrica de docs/02 §2 (tuteo, empatía, frases directas y profundidad intermedia). En la verificación no había clave de Gemini, así que las pruebas usaron el proveedor demo.

Si te preguntan…

¿Qué es la temperatura?

Un parámetro del modelo de lenguaje: con un valor bajo, el texto es predecible; con uno alto, más variado. El Redactor usa 0,8, para que suene natural sin perder la estructura. Está en el encabezado de su archivo de prompt y se cambia sin tocar código.

Si el prompt ya tiene las reglas, ¿por qué no está en “Cumple”?

Porque una regla escrita no garantiza el resultado. Con el proveedor demo los textos son plantillas; el tono real solo se puede juzgar leyendo capítulos escritos por Gemini.

REQUERIMIENTO NO FUNCIONAL · 2 DE 4

RNF-02

Menos de 60 segundos por capítulo

Implementado; falta medirlo con el modelo real

QUÉ PIDE EL DOCUMENTO

La generación individual de cada capítulo no debe exceder un tiempo razonable de procesamiento (ej. menos de 60 segundos por capítulo) para mantener un flujo de trabajo fluido durante las pausas de validación.

TU TURNO · DILO ASÍ

El texto aparece en vivo mientras se escribe, con el avance por secciones y el tiempo estimado. En el modo demo, el capítulo 1 tarda unos 7,2 segundos. Con el backend, cada capítulo tiene un tiempo límite de 60 segundos; falta medir el tiempo real con Gemini.

Cómo lo cumple

- Streaming: las palabras aparecen poco a poco, con un cursor, como si alguien escribiera.
- A la izquierda se ve en qué sección va el Redactor y la bitácora de los agentes; arriba, el tiempo estimado (“falta menos de un minuto”).
- Puedes leer los capítulos terminados con “Leer ahora” mientras se escriben los demás, o salir: el trabajo sigue.
- Medido en el modo demo con la velocidad normal: capítulo 1 en 7,2 s.
- Con el backend: streaming por SSE desde el primer trozo de texto, tiempo límite de 60 s por intento, un reintento automático y después un error con “Reintentar” que no pierde lo escrito. Con el proveedor demo, un libro de seis capítulos tarda unos 25 s.

Cómo se comprueba

`RNF-02 · Cada capítulo se genera en menos de 60 segundos, con streaming`

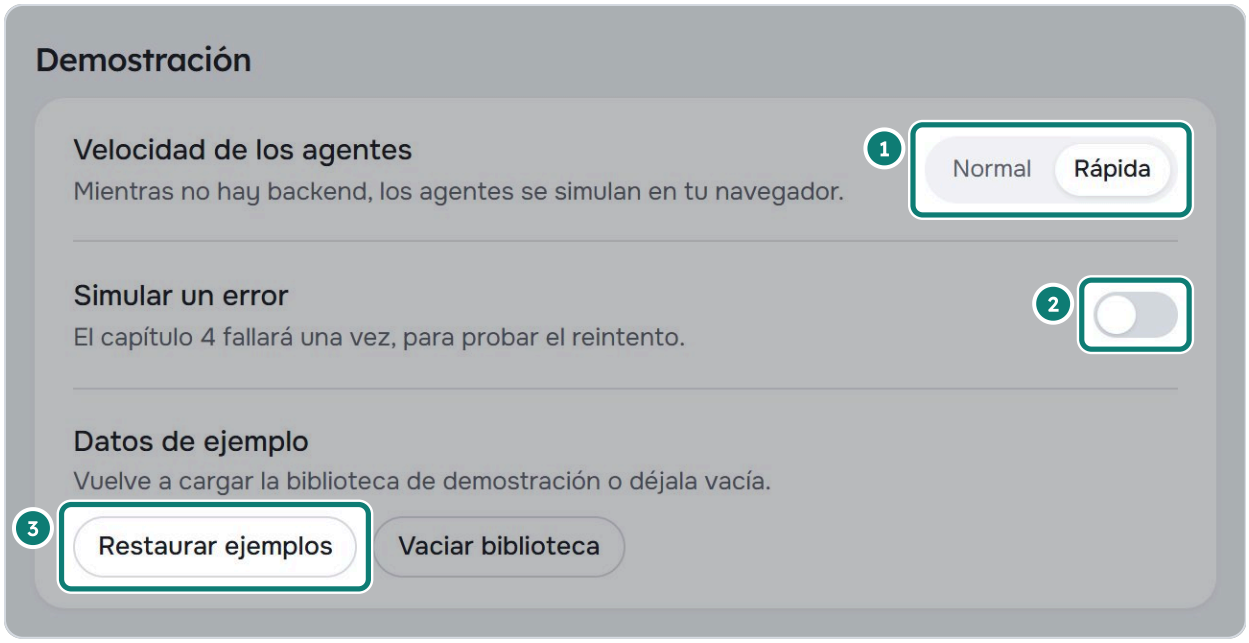
Mide el capítulo 1 con la velocidad normal y comprueba que el texto crece mientras se escribe. En el backend, `test_reintento_automatico_por_tiempo_limite` fuerza el tiempo límite y `test_eventos_sse` comprueba el streaming.

Prueba superada | Capítulo 1 en **7,2 s** | El texto **crece** durante la escritura

Dónde verlo, paso a paso

1 **Pon la velocidad en Normal**

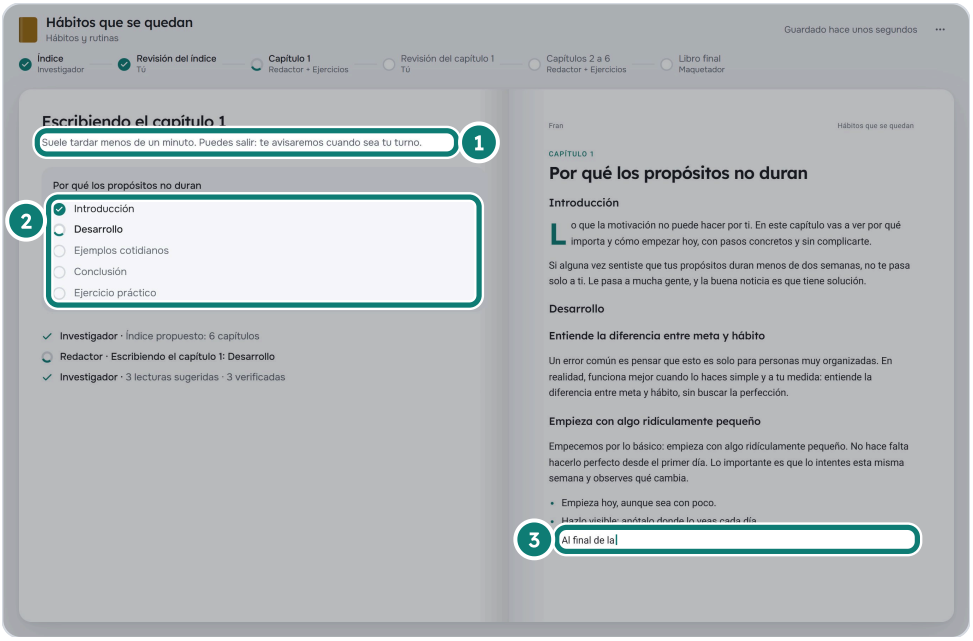
En Ajustes → Demostración. Con “Normal” ves el tiempo de referencia; “Rápida” sirve para demostrar sin esperar.



1. Velocidad de los agentes: Normal para medir tiempos; Rápida para demostrar.
2. Simular un error: el capítulo 4 falla una vez, para probar “Reintentar”.
3. Restaurar ejemplos: vuelve a cargar la biblioteca de demostración.

2 Aprueba un índice y mira el capítulo 1

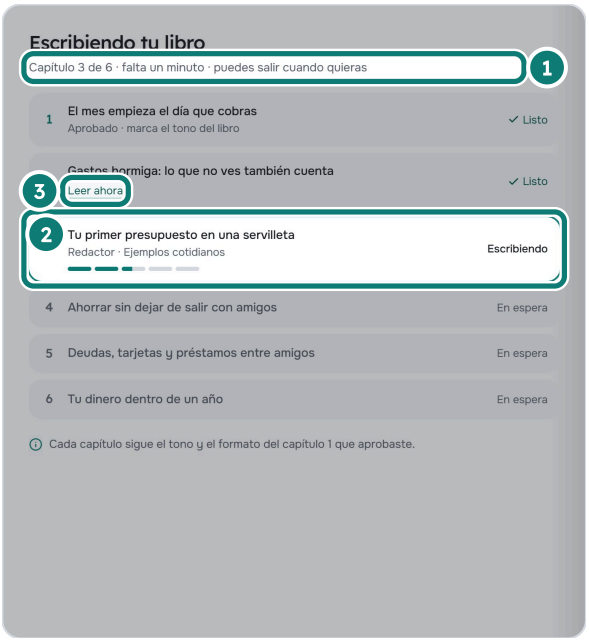
Crea un ebook, aprueba el índice y mira las dos mitades de la pantalla.



1. “Suele tardar menos de un minuto”: lo avisa desde el principio.
2. Las secciones se marcan a medida que el Redactor avanza.
3. El texto aparece en vivo: esta es la línea que se está escribiendo, con el cursor al final.

3 Aprueba el capítulo 1 y mira el resto

Toca “Aprobar y escribir el resto”.



1. Qué capítulo va y cuánto falta, aproximadamente.
2. El avance del capítulo en curso, sección por sección.
3. “Leer ahora”: los capítulos terminados se pueden leer mientras se escriben los demás.

Dónde está en el código

`src/lib/api/mock/simulator.ts:169`	Modo demo: streaming del texto por fragmentos
`src/components/workspace/stages/Writing.tsx:30`	Tiempo estimado
`src/components/workspace/previews/ChapterScroller.tsx:8`	Lectura en vivo del capítulo
`backend/app/config.py:59`	Con backend: 60 s por capítulo
`backend/app/graph/nodes.py:410`	Un reintento automático y luego un error reintentable

Qué falta medir

Medir el tiempo real con Gemini. La meta es que el 95 % de los capítulos tarde menos de 60 segundos (docs/02 §2).

Si te preguntan…

¿Por qué el modo demo es tan rápido?

Está calibrado para demostrar el flujo, no para medir. Por eso este requerimiento sigue como “Implementado; falta medirlo con el modelo real” hasta tomar tiempos con Gemini.

¿Y si un capítulo falla?

Aparece un error con “Reintentar” y no se pierde lo escrito. En el modo demo puedes probarlo en Ajustes → Demostración → Simular un error: el capítulo 4 falla una vez.

¿Cómo llega el texto en vivo desde el backend?

Por SSE (*Server-Sent Events*): el servidor envía eventos `token` por un solo canal por usuario y la interfaz los agrega al capítulo, igual que en el modo demo (D-40).

¿Y si Gemini tarda más de 60 segundos?

El intento se corta, se reintenta una vez y, si vuelve a pasar, aparece “No pudimos terminar el capítulo” con “Reintentar”. Para no alargar la espera, a Gemini se le pide un razonamiento corto.

REQUERIMIENTO NO FUNCIONAL · 3 DE 4

RNF-03

Consistencia y cohesión textual

Implementado; falta medirlo con el modelo real

QUÉ PIDE EL DOCUMENTO

Los agentes redactores deben mantener coherencia terminológica y de estilo a lo largo de los 5 a 7 capítulos sin reiteraciones redundantes ni contradicciones entre secciones.

TU TURNO · DILO ASÍ

El capítulo 1 que apruebas es la referencia de tono y formato para todo el libro, y los capítulos se escriben uno detrás de otro para que cada uno conozca a los anteriores.

Cómo lo cumple

- La interfaz lo dice dos veces: en la pausa (“Este capítulo marcará el tono y el formato de todo el libro”) y durante la redacción (“Cada capítulo sigue el tono y el formato del capítulo 1 que aprobaste”).
- La redacción es secuencial: nunca hay dos capítulos escribiéndose a la vez. Los demás aparecen “En espera”.
- Todos los capítulos comparten la estructura de RF-03 y los títulos del índice aprobado.
- Con el backend, al aprobar el capítulo 1 se extrae una guía de estilo. Cada capítulo recibe el índice, el *brief*, esa guía, el capítulo 1 completo como referencia y el resumen y la conclusión de los anteriores, con la orden de no repetir ni contradecir.

Cómo se comprueba

`RNF-03 · El capítulo 1 marca el tono y el resto se escribe en orden`

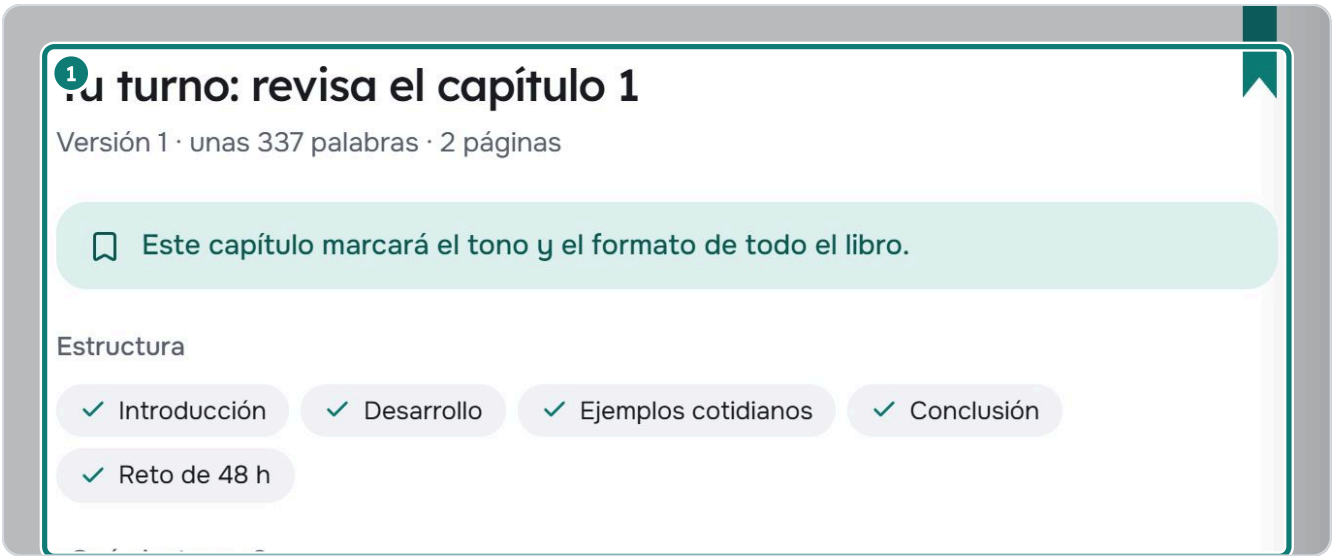
Muestrea el estado durante toda la redacción y exige que nunca haya dos capítulos escribiéndose a la vez.

Prueba superada | Como máximo **1** capítulo a la vez | Aviso de tono en la pausa y en la redacción

Dónde verlo, paso a paso

1 **Lee el aviso de la segunda pausa**

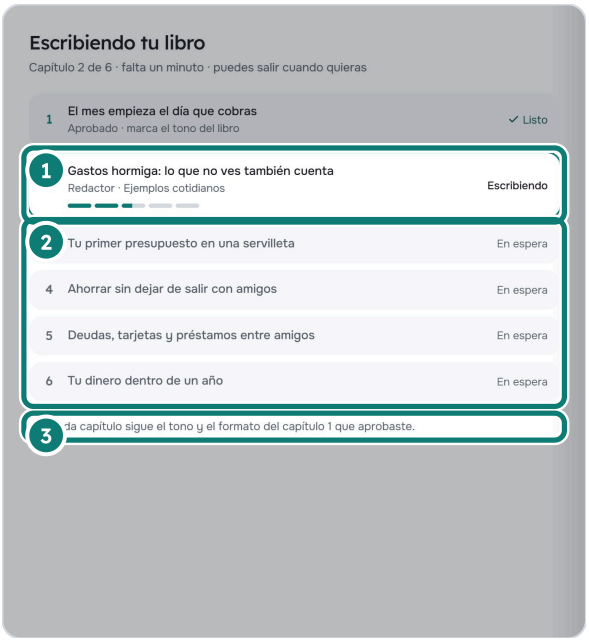
En la revisión del capítulo 1, justo debajo del título.



1 La pausa 2 avisa que este capítulo será la referencia de tono y formato para todo el libro.

2 **Aprueba y mira el orden**

Toca “Aprobar y escribir el resto” y mira la lista de capítulos.



1 Solo un capítulo se escribe a la vez…

2 …los demás esperan su turno.

3 El recordatorio: cada capítulo sigue el tono y el formato del capítulo 1.

Dónde está en el código

`src/lib/api/mock/simulator.ts:204`	Modo demo: escritura secuencial de los capítulos 2 en adelante
`src/components/workspace/stages/ChapterOneReview.tsx:101`	Aviso en la pausa 2
`src/components/workspace/stages/Writing.tsx:128`	Nota durante la redacción
`backend/app/graph/nodes.py:592`	Con backend: guía de estilo del capítulo 1 aprobado
`backend/app/agents/redactor.py:188`	El capítulo 1 viaja como referencia en cada llamada
`backend/app/prompts/redactor/capitulo.md:43`	No repetir ni contradecir a los anteriores

Qué falta medir

Leer un libro completo escrito por Gemini y buscar repeticiones, contradicciones y cambios de términos entre capítulos.

Si te preguntan…

¿No sería más rápido escribir en paralelo?

Sí, pero cada capítulo perdería el contexto de los anteriores y habría más repeticiones y contradicciones. Aquí se prioriza la coherencia (D-09).

¿Cómo se evitan las repeticiones con la IA real?

El Redactor recibe el resumen y la conclusión de los capítulos ya escritos y la guía de estilo del capítulo 1, con la instrucción de no repetir ideas, ejemplos ni personas. Falta comprobarlo leyendo un libro escrito por Gemini.

REQUERIMIENTO NO FUNCIONAL · 4 DE 4

Cumple

RNF-04

Arquitectura modular de cuatro agentes

QUÉ PIDE EL DOCUMENTO

El backend del sistema debe estar estructurado de forma modular (dividiendo funciones entre Agente Investigador, Agente Redactor, Agente de Ejercicios y Agente Maquetador/Exportador) permitiendo la fácil sustitución o reajuste de los prompts de cada rol.

TU TURNO · DILO ASÍ

El backend tiene cuatro agentes, cada uno en su módulo, y cada tarea tiene su prompt en un archivo propio. Para cambiar cómo escribe el Redactor editas su archivo, sin tocar código. Y el modelo de lenguaje se cambia con una variable.

Cómo lo cumple

- Un módulo por agente: `investigador.py`, `redactor.py`, `ejercicios.py` y `maquetador.py`, en `backend/app/agents/`, orquestados por un grafo de LangGraph.
- Prompts en archivos independientes, uno por tarea (`backend/app/prompts/<agente>/<tarea>.md`), con la temperatura y el máximo de tokens en el encabezado. Se recargan al editarlos, sin reiniciar el servidor.
- El modelo de lenguaje es intercambiable: Gemini por defecto, Claude, OpenAI o un proveedor demo sin conexión, detrás de `backend/app/llm/`. Se elige con `LLM_PROVIDER` y los agentes no cambian (D-34).
- La interfaz muestra al responsable de cada fase (Investigador, Redactor + Ejercicios, Maquetador y "Tú" en las pausas) y la bitácora dice qué hace cada agente, en el modo demo y con el backend.

Cómo se comprueba

`RNF-04 · Los cuatro agentes trabajan por separado y se ven`

En el frontend, comprueba que las fases muestran a Investigador, Redactor + Ejercicios y Maquetador, y que en la bitácora participan exactamente los cuatro agentes. En el backend, `test_flujo_completo` recorre el grafo entero, `test_prompts_tienen_reglas_explicitas` y `test_formato_de_prompt` validan los archivos de prompt, y `test_llm.py` y `test_llm_sdks.py` prueban cada proveedor.

Prueba superada · **4** agentes en la bitácora · Fases con su responsable · **147 de 147** pruebas del backend

Dónde verlo, paso a paso

1 Lee la barra de fases

Está arriba en cualquier libro. Cada fase dice quién trabaja.



1. Agente Investigador: propone el índice.
2. Tú: las dos pausas de aprobación.
3. Agente Redactor y Agente de Ejercicios: escriben los capítulos.
4. Agente Maquetador: arma el libro final.

2 Mira la bitácora mientras trabajan

Mientras el Investigador o el Redactor trabajan, a la izquierda aparece la actividad de los agentes.



1. Cada línea dice qué agente trabajó y qué hizo: aquí, el Investigador (el índice y las lecturas sugeridas)…
2. …y ahora el Redactor, escribiendo el capítulo 1.

3 Abre las carpetas del backend

En el repositorio, `backend/app/agents/` tiene un módulo por agente y `backend/app/prompts/`, un archivo por tarea. Así están hoy:

BACKEND/APP · UN MÓDULO POR AGENTE Y UN PROMPT POR TAREA

Orquestador

`graph/builder.py`

`orquestador/guardarrail.md`
temperatura 0

Investigador

`agents/investigador.py`

`investigador/capitulo_alternativo.md`
temperatura 0,8

`investigador/indice.md` temperatura 0,7

`investigador/lecturas.md`
temperatura 0,2

Redactor

`agents/redactor.py`

`redactor/capitulo.md`
temperatura 0,8

`redactor/guia_estilo.md`
temperatura 0,3

Ejercicios

`agents/ejercicios.py`

`ejercicios/ejercicio.md`
temperatura 0,7

Maquetador

`agents/maquetador.py`

`maquetador/contraportada.md`
temperatura 0,7

Dónde está en el código

`backend/app/graph/builder.py:31`	Grafo de LangGraph: guardarraíl, agentes y pausas
`backend/app/agents/investigador.py:113`	Agente Investigador
`backend/app/agents/redactor.py:208`	Agente Redactor
`backend/app/agents/ejercicios.py:103`	Agente de Ejercicios
`backend/app/agents/maquetador.py:65`	Agente Maquetador/Exportador
`backend/app/prompts/__init__.py:94`	Recarga un prompt cuando cambia su archivo
`backend/app/llm/base.py:123`	Capa intercambiable del modelo de lenguaje
`src/lib/api/index.ts:12`	Fachada del frontend: modo demo o backend

Si te preguntan…

¿Qué significa cambiar un prompt “fácilmente”?

Cada tarea de cada agente lee su prompt de un archivo propio, con la temperatura en el encabezado. Para ajustar el tono del Redactor editas `redactor/capitulo.md`: no tocas el código ni a los otros agentes, y el servidor lo recarga solo.

¿Y si mañana queremos otro modelo?

Se cambia `LLM_PROVIDER` en `backend/.env` (gemini, anthropic, openai o demo). Los agentes solo conocen la interfaz `LLMProvider`, así que no se tocan.

¿Qué es LangGraph?

Una librería de Python para construir flujos de agentes como un grafo de pasos, con pausas (`interrupt()`) y estado guardado. Folio la usa para las dos pausas de RF-05 y para retomar el trabajo si el servidor se reinicia.

¿El sitio publicado usa estos agentes?

No. GitHub Pages solo sirve archivos estáticos, así que publica el modo demo, con los agentes simulados en el navegador. Los agentes reales corren con el backend en local; para publicarlo hace falta un servidor con Python y Chromium (P-08).

Folio

PARTE III

Requerimientos de accesibilidad

Para que cualquier persona pueda leer el libro: encabezados para lectores de pantalla, lenguaje claro y un PDF con buen contraste.

RA-01	Jerarquía H1, H2 y H3 en Markdown y PDF	49
RA-02	Lenguaje claro y comprensible	53
RA-03	Contraste y tipografía limpia en el PDF	56

REQUERIMIENTO DE ACCESIBILIDAD · 1 DE 3

Cumple

RA-01

Jerarquía H1, H2 y H3 en Markdown y PDF

QUÉ PIDE EL DOCUMENTO

Los PDFs y archivos Markdown exportados deben mantener una jerarquía clara de encabezados (H1, H2, H3) para garantizar la lectura adecuada a través de lectores de pantalla (screen readers).

TU TURNO · DILO ASÍ

El libro exportado usa tres niveles de encabezado, sin saltos: H1 para el título y los capítulos, H2 para las secciones y H3 para los subtemas. El PDF va etiquetado para lectores de pantalla.

Cómo lo cumple

- Markdown: `#` para el título, cada capítulo, las Referencias y "Sobre este libro"; `##` para las secciones y `###` para los subtemas.
- PDF etiquetado: tiene árbol de estructura (`StructTreeRoot`), marcadores, idioma `es` y exactamente los niveles H1, H2 y H3.
- En el editor, si alguien escribe `#` o `##` a mano, se convierte en `###` para no romper la jerarquía.
- Con el backend, el código hace lo mismo con el texto del modelo, y el Agente Maquetador genera en el servidor el mismo Markdown y un PDF etiquetado con marcadores.

Cómo se comprueba

`RA-01 · Jerarquía H1/H2/H3 en Markdown y PDF etiquetado`

Recorre los encabezados del Markdown (sin saltos, máximo nivel 3, un H1 por capítulo), lee las etiquetas del PDF y comprueba la conversión del editor. En el backend, `test_flujo_completo` hace lo mismo con los archivos del servidor y `test_normalize_markdown_ra01`, con el texto del modelo.

Prueba superada | Markdown: **9** H1 · **32** H2 · **18** H3 | PDF: etiquetas **H1, H2, H3** | `#` → `###` en el editor

Dónde verlo, paso a paso

1 Descarga el Markdown y mira los

Cada # es un nivel de encabezado: # es H1, ## es H2 y ### es H3.

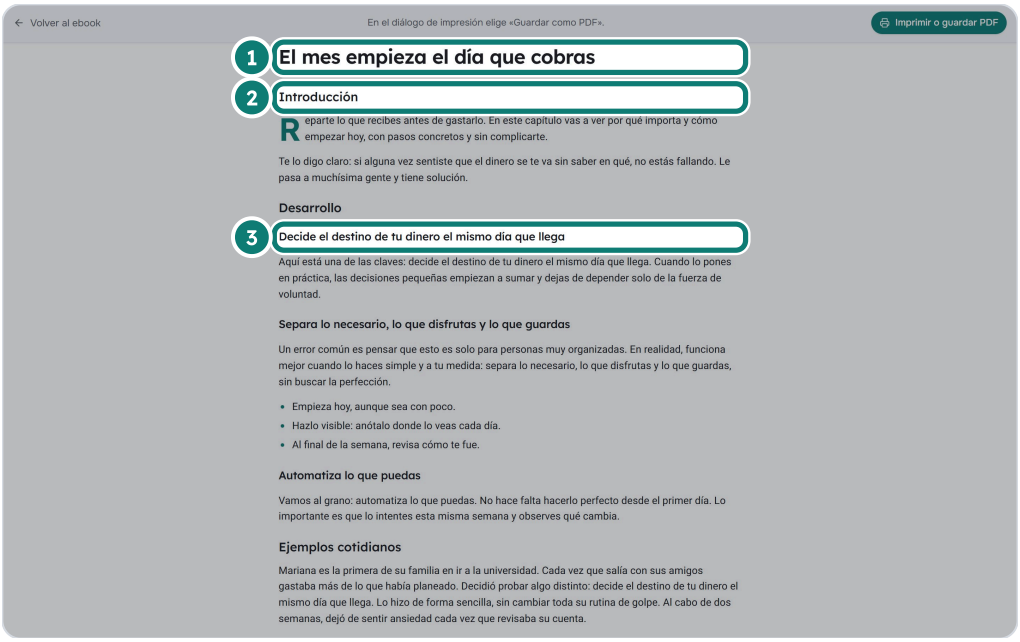
ASÍ EMPIEZAN LOS ENCABEZADOS DEL MARKDOWN EXPORTADO

- H1 `# Ahorra sin dejar de vivir`
  - H2 `## Aviso legal`
  - H2 `## Índice`
- H1 `# Capítulo 1: El mes empieza el día que cobras`
  - H2 `## Introducción`
  - H2 `## Desarrollo`
    - H3 `### Decide el destino de tu dinero el mismo día que llega`
    - H3 `### Separa lo necesario, lo que disfrutas y lo que guardas`
    - H3 `### Automatiza lo que puedas`
  - H2 `## Ejemplos cotidianos`
  - H2 `## Conclusión`
  - H2 `## Ejercicio práctico`

… y así hasta el capítulo 6, más Referencias y “Sobre este libro”: 9 H1, 32 H2 y 18 H3 en total, sin saltar niveles.

2 Abre la vista de impresión

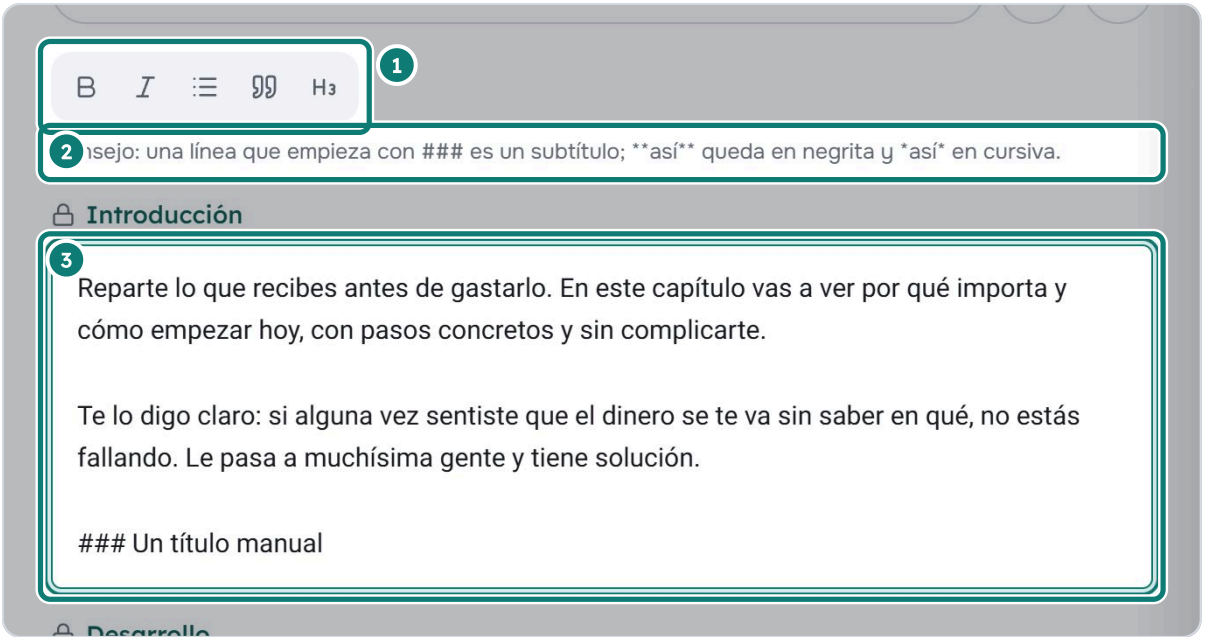
Es lo que se guarda como PDF: los mismos tres niveles, y el PDF sale etiquetado.



1. H1: el título de cada capítulo.
2. H2: cada sección (Introducción, Desarrollo…).
3. H3: los subtemas dentro del Desarrollo.

3 Intenta romper la jerarquía

En la revisión final, abre “Texto” y escribe “# Un título manual” al final de la Introducción.



1. La barra de formato solo ofrece subtítulo H3 (más negrita, cursiva, lista y cita).
2. El consejo recuerda que ### es un subtítulo.
3. Escribiste “# Un título manual” y quedó “### Un título manual”: no rompe la jerarquía.

Dónde está en el código

`src/lib/markdown.ts:32`	Capítulos como H1 en el Markdown
`src/components/workspace/stages/TextEditor.tsx:13`	Convierte # y ## en ###
`src/app/imprimir/page.tsx:125`	Modo demo: título y capítulos en H1 en el PDF
`backend/app/agents/redactor.py:246`	Con backend: convierte los # y ## del modelo en ###
`backend/app/export/pdf.py:232`	Lee las etiquetas del PDF del servidor en las pruebas

Si te preguntan…

¿Qué es un PDF etiquetado?

Un PDF que, además del aspecto de cada página, guarda su estructura: qué es título, qué es párrafo y en qué orden se lee. Es lo que usan los lectores de pantalla.

¿Por qué no usar H4?

El requerimiento pide H1 a H3 y con tres niveles basta para un libro corto. Más niveles complican la navegación con lector de pantalla.

¿Cómo lo compruebo sin lector de pantalla?

En Acrobat Reader, abre el panel de Etiquetas (Ver → Mostrar/ocultar → Paneles laterales → Etiquetas). O abre el Markdown y cuenta los #.

¿Lo probaron con un lector de pantalla?

Todavía no con NVDA: las pruebas leen las etiquetas del PDF de forma automática. Revisarlo con un lector de pantalla real es uno de los pasos que faltan.

REQUERIMIENTO DE ACCESIBILIDAD · 2 DE 3

RA-02

Lenguaje claro y comprensible

Implementado; falta medirlo con el modelo real

QUÉ PIDE EL DOCUMENTO

El contenido redactado debe utilizar una sintaxis directa y explicaciones simples para garantizar un nivel de lecturabilidad accesible a un público joven o diverso sin formación técnica previa.

TU TURNO · DILO ASÍ

Medimos la legibilidad del libro con INFLESZ, una escala hecha para textos en español: salió 78,8, “bastante fácil”. La prueba exige 55 o más. Falta repetir la medición sobre un libro escrito por Gemini.

Cómo lo cumple

- El libro de prueba (modo demo) obtiene INFLESZ 78,8 (“bastante fácil”), con 9,1 palabras por frase y 1,9 sílabas por palabra, sobre 2.305 palabras.
- En la escala INFLESZ, de 55 a 65 es “normal” y de 65 a 80, “bastante fácil”: el texto queda por encima de lo normal.
- Frases cortas, segunda persona, sin tecnicismos y con ejemplos de la vida diaria.
- Con el backend, el prompt del Redactor pide frases de menos de 25 palabras, una idea por párrafo, vocabulario cotidiano y explicar en la misma frase cualquier término técnico.

Cómo se comprueba

`RA-02 · Lenguaje claro (índice de legibilidad INFLESZ)`

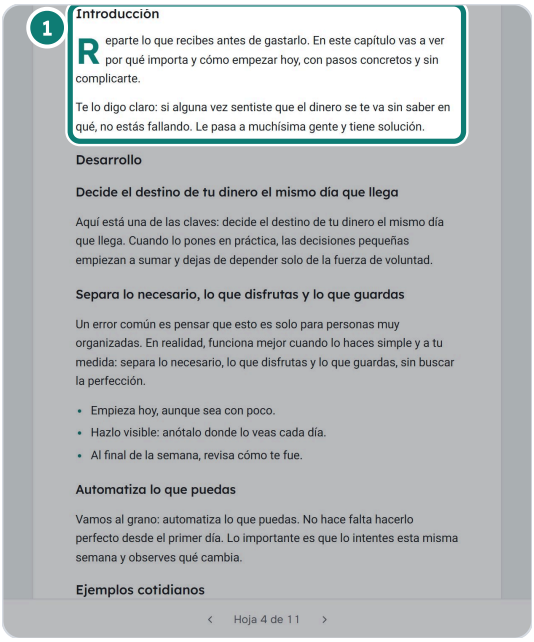
Calcula el índice sobre el texto del libro exportado (sin encabezados) y exige 55 o más.

Prueba superada | INFLESZ **78,8** | **9,1** palabras por frase | **1,9** sílabas por palabra | **253** frases analizadas

Dónde verlo, paso a paso

1 Lee un párrafo en voz alta

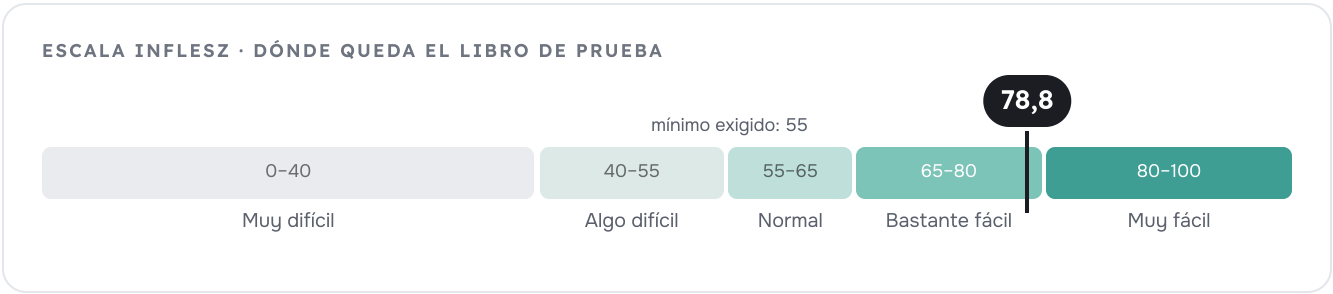
Abre un capítulo de “Ahorra sin dejar de vivir”.



1 Frases cortas, de tú a tú y sin tecnicismos: sobre este texto se calcula la legibilidad.

2 Muestra la medición

La prueba calcula el índice INFLESZ sobre todo el libro exportado, sin los títulos: 78,8.



Dónde está en el código

`e2e/legibilidad.ts:29`	Cálculo del índice INFLESZ
`backend/app/prompts/redactor/capitulo.md:12`	Con backend: reglas de lenguaje claro en el prompt

Qué falta medir

Calcular el mismo índice INFLESZ sobre un libro escrito por Gemini (mínimo 55).

Si te preguntan…

¿Qué es INFLESZ?

Una escala de legibilidad para español (Barrio-Cantalejo y colaboradores, 2008) basada en la fórmula de perspicuidad de Szigriszt-Pazos. Cuenta sílabas por palabra y palabras por frase: cuanto más alto, más fácil.

¿Una fórmula basta para decir que un texto es claro?

No del todo: mide la forma, no si la explicación se entiende. Por eso se complementa con el tono de RNF-01 y con la revisión de la persona en las pausas.

REQUERIMIENTO DE ACCESIBILIDAD · 3 DE 3

Cumple

RA-03

Contraste y tipografía limpia en el PDF

QUÉ PIDE EL DOCUMENTO

El archivo PDF generado debe emplear fondos neutros (preferiblemente blanco) con texto en alto contraste (negro/gris oscuro) y fuentes tipográficas legibles (ej. Arial, Helvetica o Roboto) en tamaños mínimos de 11pt para el cuerpo de texto.

TU TURNO · DILO ASÍ

El PDF siempre sale con hoja blanca, texto casi negro y Roboto, Arial o Helvetica desde 11 pt: no hay forma de elegir menos. Y si eliges un color de portada con poco contraste, Folio te avisa y lo corrige.

Cómo lo cumple

- “Portada y estilo” solo ofrece Roboto, Arial y Helvetica, y tamaños de 11, 12 y 13 pt.
- La hoja del libro es siempre blanca, también con el tema oscuro de la app.
- Color de tela personalizado: si el texto blanco queda por debajo de 4,5:1, aparece un aviso y “Usar el tono accesible”, que oscurece el color hasta cumplir.
- Medido en la vista de impresión: cuerpo de 14,67 px (11 pt) en Roboto, texto #1B1D23 sobre blanco, contraste 16,8:1.
- Con el backend, la API rechaza una tela sin contraste y sube sola el velo de la imagen hasta el mínimo, y el PDF del servidor usa las mismas letras y tamaños, con las fuentes incrustadas.

Cómo se comprueba

`RA-03 · Fondo blanco, alto contraste, letra legible y 11 pt como mínimo`

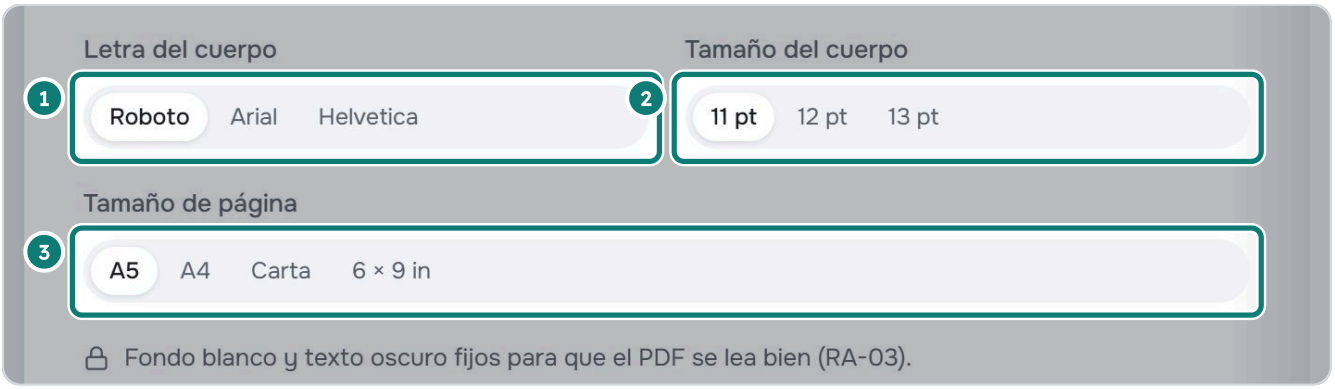
Comprueba las opciones permitidas, el aviso y la corrección del contraste, y mide tamaño, fuente, fondo y contraste reales en la vista de impresión. En el backend, `test_validaciones_del_diseno` envía una tela sin contraste (recibe el tono sugerido) y un cuerpo de 10 pt (lo rechaza).

Prueba superada · **14,67 px** = 11 pt · **Roboto** · Fondo **blanco** · Contraste **16,8:1**

Dónde verlo, paso a paso

1 Abre Portada y estilo

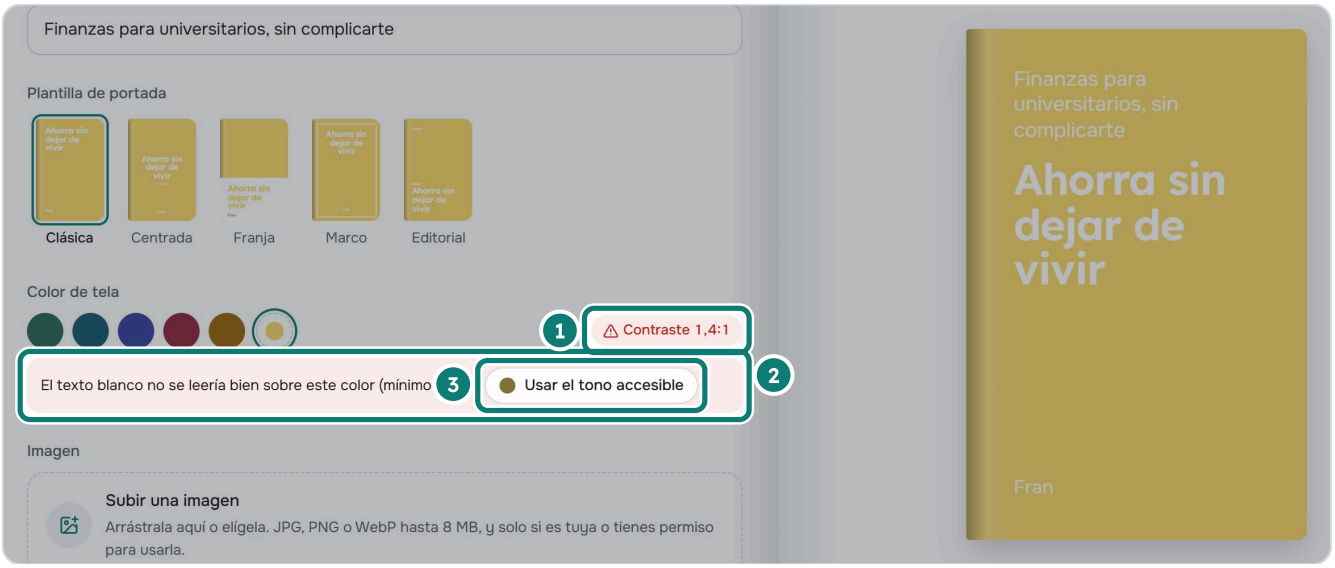
En un libro listo, abre la pestaña “Portada y estilo” y elige la vista “Interior”.



1. Letra del cuerpo: solo Roboto, Arial o Helvetica.
2. Tamaño del cuerpo: 11, 12 o 13 pt. No existe una opción menor de 11.
3. Tamaño de página: A5, A4, Carta o 6 × 9 in.

2 Elige un color con poco contraste

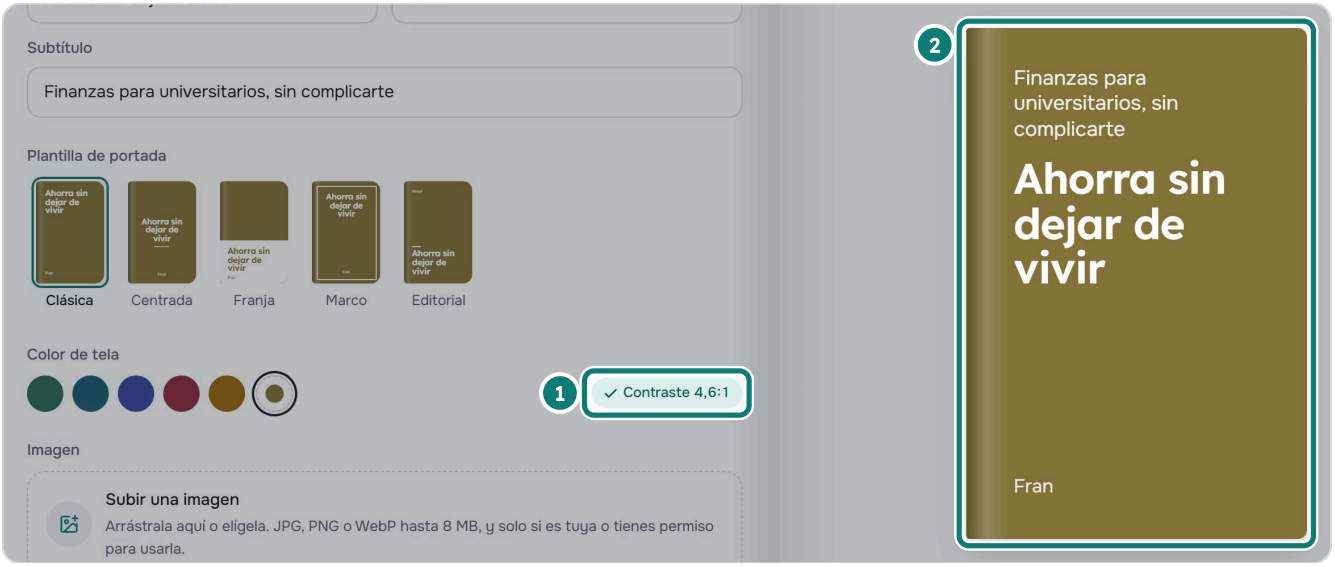
Vuelve a la vista “Portada”. En “Color de tela”, abre el color personalizado y elige un amarillo claro.



1. Con un amarillo claro, el indicador marca “Contraste 1,4:1”…
2. …Folio avisa que el texto blanco no se leería bien (mínimo 4,5:1)…
3. …y ofrece “Usar el tono accesible”.

3 Usa el tono accesible

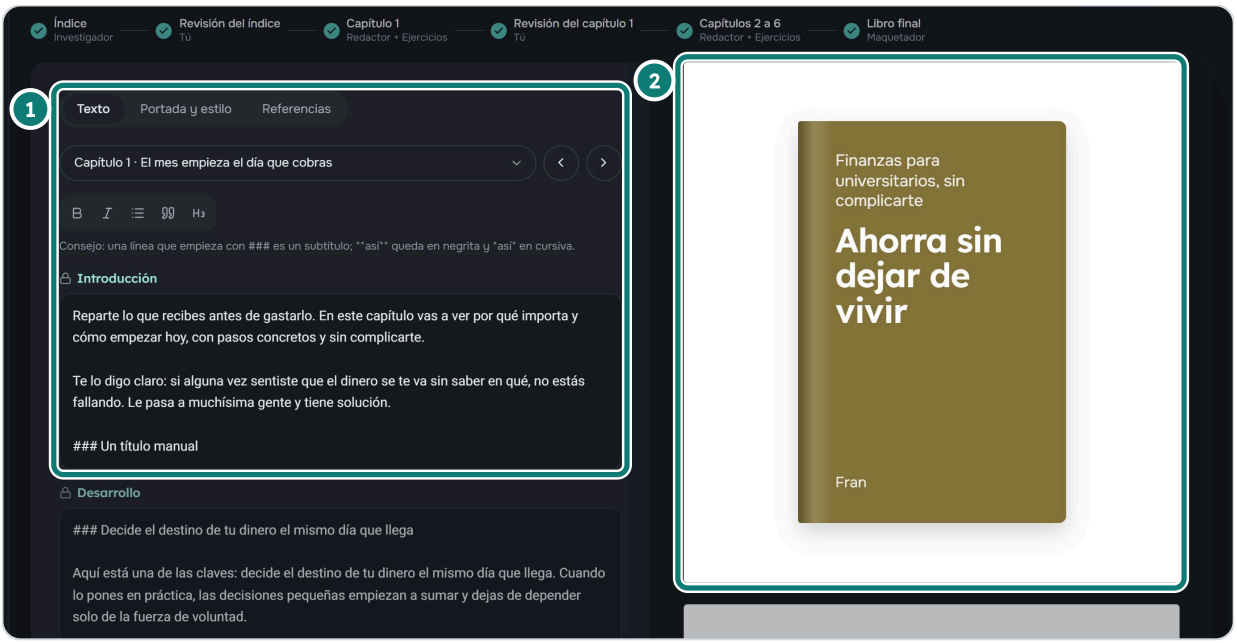
Toca “Usar el tono accesible”.



1. El color se oscurece hasta cumplir: el indicador pasa a verde.
2. La portada se actualiza al instante con el tono accesible.

4 Cambia al tema oscuro

En Ajustes → Preferencias → Tema, elige “Oscuro” y vuelve al libro.



1. Con el tema oscuro cambia la app…
2. …pero la hoja del libro sigue blanca, como saldrá en el PDF.

Dónde está en el código

`src/lib/design.ts:69`	Letras permitidas
`src/lib/design.ts:70`	Tamaños permitidos: 11, 12 y 13 pt
`src/lib/design.ts:118`	Tono accesible (4,5:1 como mínimo)
`src/components/workspace/stages/DesignPanel.tsx:95`	Validación del contraste en vivo
`src/app/globals.css:178`	Hoja del libro siempre blanca
`backend/app/design.py:46`	Con backend: el mismo tono accesible en el servidor
`backend/app/api/ebooks.py:571`	La API valida letra, tamaño, tela e imagen

Si te preguntan…

¿Qué significa 4,5:1?

Es la relación de luminancia entre el texto y el fondo. WCAG pide al menos 4,5:1 para texto normal. El cuerpo del libro tiene 16,8:1; el máximo posible, negro sobre blanco, es 21:1.

¿Por qué 14,67 px son 11 pt?

Un punto es 1/72 de pulgada y un píxel CSS, 1/96. Entonces 11 pt × 96 ÷ 72 = 14,67 px.

¿Y el tema oscuro?

Cambia la app, no el libro: la hoja siempre es blanca, porque así saldrá el PDF.

Folio

PARTE IV

Requerimientos éticos y de responsabilidad

El uso responsable del contenido: originalidad, avisos legales en temas sensibles y ejemplos sin estereotipos.

RÉR-01	Originalidad y prevención de plagio	61
RÉR-02	Aviso legal en temas sensibles	64
RÉR-03	Mitigación de sesgos y estereotipos	69

REQUERIMIENTO ÉTICO Y DE RESPONSABILIDAD · 1 DE 3

RÉR-01

Originalidad y prevención de plagio

Implementado; falta medirlo con el modelo real

QUÉ PIDE EL DOCUMENTO

Los agentes deben incluir instrucciones explícitas para generar contenido sintético original, evitando la copia textual de fuentes protegidas por derechos de autor durante la etapa de investigación.

TU TURNO · DILO ASÍ

Cada exportación declara “Contenido original generado con asistencia de IA”. Con el backend, el prompt del Redactor prohíbe copiar e inventar citas, y si Gemini detecta que una respuesta se parece a un texto existente, se pide otra. Falta revisar por muestreo un libro escrito por Gemini.

Cómo lo cumple

- En el modo demo, el contenido sale de plantillas escritas para el proyecto; no se copia de ninguna fuente externa.
- Transparencia: el diálogo de exportación, el Markdown y el PDF incluyen la nota “Contenido original generado con asistencia de IA”, y el `.md` y los metadatos del PDF llevan una marca legible por máquina (D-39).
- Con el backend, el Redactor tiene instrucciones explícitas: sus propias palabras, sin copiar ni parafrasear de cerca, sin inventar estudios, cifras ni citas y sin atribuir frases a personas reales.
- Si Gemini corta una respuesta por `RECITATION` (parecido con un texto existente), se pide otra versión. Y las lecturas que sugiere el Investigador se verifican en Crossref y Open Library (RD-09).

Cómo se comprueba

`RÉR-01 · Contenido original y nota de transparencia`

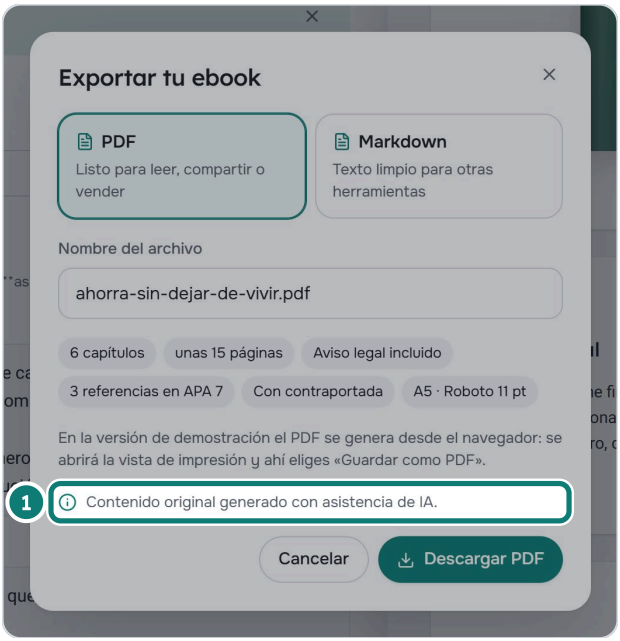
Comprueba la nota en el diálogo de exportación y al final del Markdown. En el backend, `test_prompts_tienen_reglas_explicitas` exige las reglas de originalidad en el prompt y `test_flujo_completo`, la nota y la marca de IA en los archivos.

Prueba superada · Nota en el **diálogo** · Nota al final del **Markdown**

Dónde verlo, paso a paso

1 Abre Exportar

En un libro listo, toca “Exportar”.



1. La nota de transparencia: “Contenido original generado con asistencia de IA”.

2 Mira el final del Markdown

Descarga el Markdown y baja hasta el final del archivo.

ahorra-sin-dejar-de-vivir.md

```
309  ---
310
311  *Contenido original generado con asistencia de IA.*   1
312
313  <!-- folio: generado-con-ia=true; herramienta=Folio; proveedor=demo; fecha=2026-10-20 -->   2
```

1. Al final del libro, la nota de transparencia.
2. Y una marca legible por máquina: un comentario que no se ve al leer (D-39).

Dónde está en el código

`src/components/workspace/ExportDialog.tsx:156`	Nota en el diálogo
`src/lib/markdown.ts:54`	Nota al final del Markdown
`backend/app/prompts/redactor/capitulo.md:39`	Con backend: reglas de originalidad del Redactor
`backend/app/llm/gemini.py:132`	Pide otra versión si se parece a un texto existente
`backend/app/export/markdown.py:70`	Marca de IA legible por máquina

Qué falta medir

Revisar por muestreo un libro escrito por Gemini y, si hace falta, añadir una verificación de similitud antes de exportar (opcional en docs/02 §4).

Si te preguntan…

¿Cómo se evita el plagio con un modelo real?

Con varias capas: instrucciones en el prompt del Redactor (palabras propias, nada de copiar ni de citas inventadas), el filtro `RECITATION` de Gemini, que corta una respuesta parecida a un texto existente, y las referencias verificadas para dar crédito a las fuentes. La verificación de similitud antes de exportar sigue como opcional.

¿Por qué avisar que hubo IA?

Por transparencia con quien compra o lee el libro. Forma parte de la categoría de ética y responsabilidad, y la marca legible por máquina sigue el espíritu del art. 50.2 del AI Act.

REQUERIMIENTO ÉTICO Y DE RESPONSABILIDAD · 2 DE 3

Cumple

RÉR-02

Aviso legal en temas sensibles

QUÉ PIDE EL DOCUMENTO

Todo eBook enfocado en temas como finanzas personales, gestión de emociones o autoayuda debe incluir automáticamente un aviso legal visible aclarando que el contenido es meramente educativo e informativo, y no sustituye la asesoría profesional (financiera, médica o psicológica).

TU TURNO · DILO ASÍ

Si el tema es sensible, el aviso legal se agrega solo: se anuncia desde el índice, va entre la portada y el índice del libro, y no se puede borrar.

Cómo lo cumple

- Cada nicho tiene una sensibilidad (financiera, salud emocional, salud física o autoayuda) con su propio texto de aviso.
- Se anuncia desde la primera pausa: “Incluirá un aviso legal porque el tema es financiero”, con “Ver el texto”.
- En el libro va entre la portada y el índice. En el editor aparece con candado y no se puede quitar.
- Gestión del tiempo, Organización del estudio y Primer empleo no lo llevan: son habilidades prácticas.
- Con el backend, el Investigador también clasifica la sensibilidad (nunca la baja a “ninguno” si el nicho es sensible), el Maquetador inserta el aviso y la API no deja quitarlo.

Cómo se comprueba

`RÉR-02 · El aviso legal se anuncia desde el índice`

`RÉR-02 · Aviso legal automático y no removible en temas sensibles`

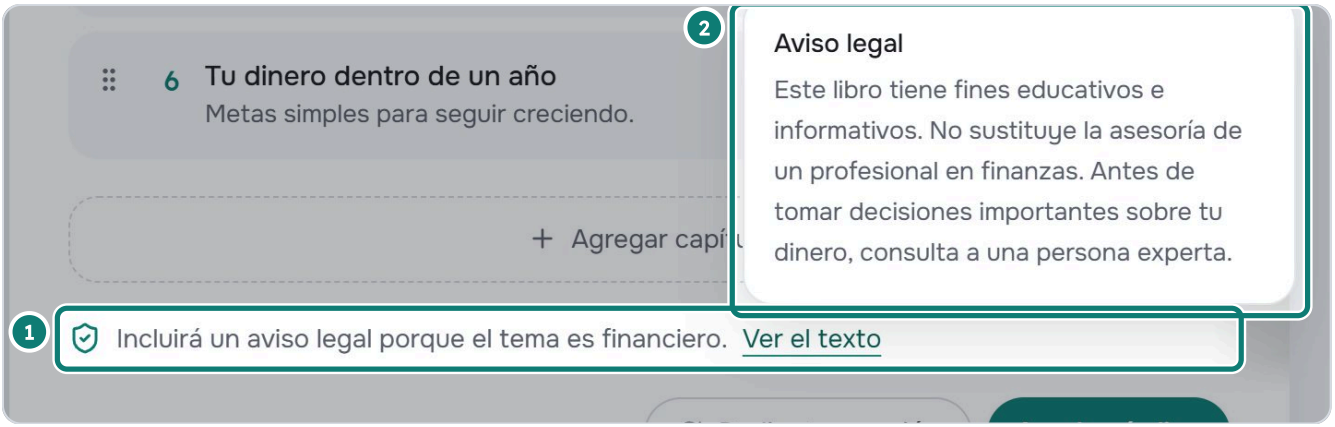
Comprueba el aviso en el índice, su posición en el Markdown (entre la portada y el índice), el bloqueo en el editor, el caso emocional y que un tema no sensible no lo lleva.

Prueba superada | Finanzas: **con aviso** | Estrés y ansiedad: **con aviso** | Gestión del tiempo: **sin aviso**

Dónde verlo, paso a paso

1 Crea un libro de finanzas

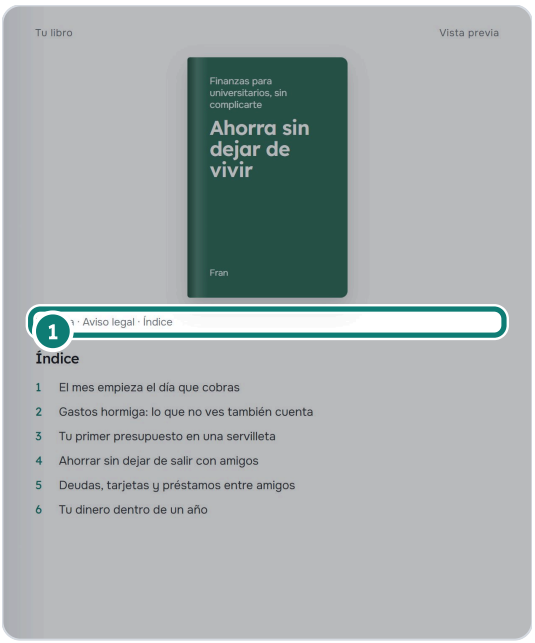
En la primera pausa, mira debajo del índice y toca “Ver el texto”.



1. Desde el índice avisa que el libro llevará aviso legal y por qué: el tema es financiero.
2. “Ver el texto” muestra el aviso completo.

2 Mira el orden del libro

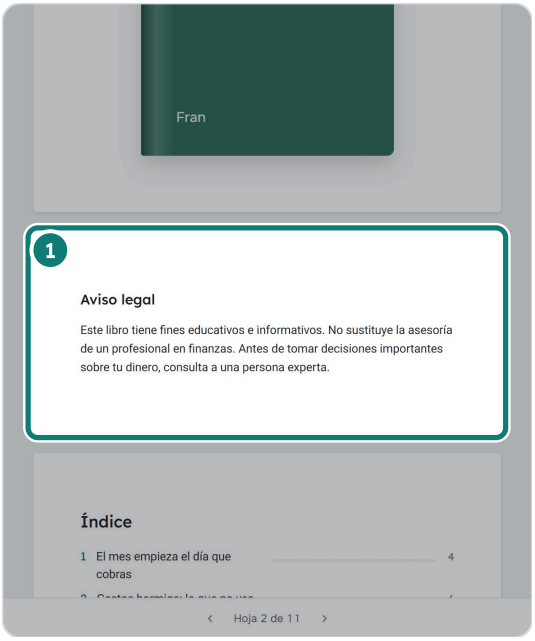
A la derecha, en “Tu libro”, debajo de la portada.



1. En “Tu libro”, el orden ya incluye el aviso: Portada · Aviso legal · Índice.

3 Ábrelo cuando esté listo

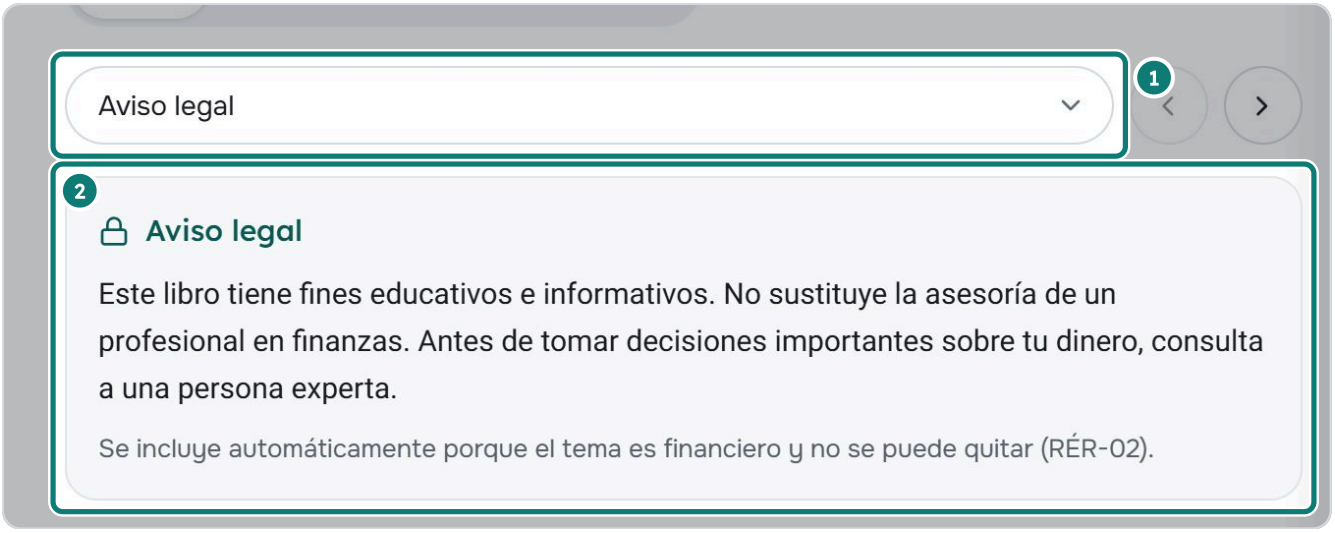
En “Tu libro”, baja una hoja después de la portada.



1. En el libro, el aviso va justo después de la portada y antes del índice.

4 Intenta borrarlo

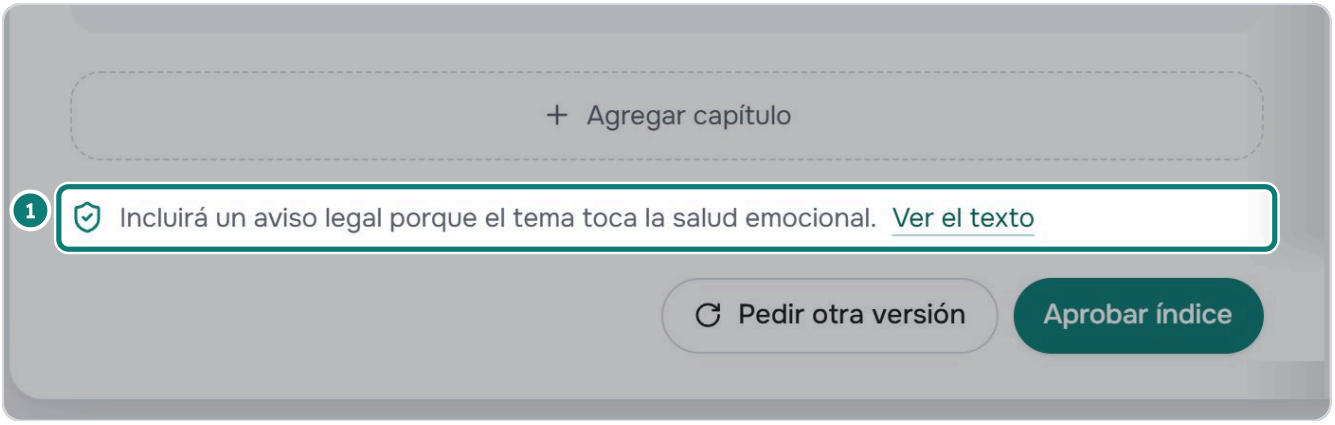
En la revisión final, abre “Texto” y elige “Aviso legal” en el selector.



1. En el selector eliges “Aviso legal”…
2. …y aparece con candado: se incluye solo y no se puede quitar.

5 Prueba con un tema emocional

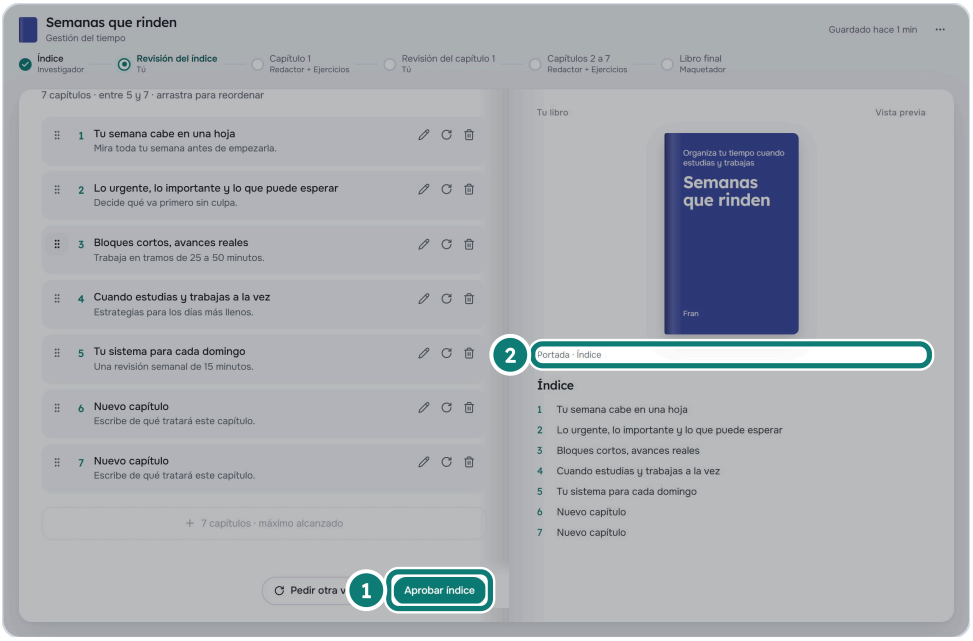
Crea un ebook con la idea “Cómo manejar la ansiedad y el estrés antes de un examen importante”.



1. Con un tema de ansiedad, el aviso es otro: “el tema toca la salud emocional”.

6 Prueba con un tema que no es sensible

Abre “Semanas que rinden” (Gestión del tiempo), que está en la primera pausa.



1. Gestión del tiempo no es un tema sensible: entre “Agregar capítulo” y los botones no aparece la línea del aviso…
2. …y el libro va de la portada directo al índice: “Portada · Índice”.

Dónde está en el código

`src/lib/guardrails.ts:34`	Textos del aviso por categoría
`src/lib/niches.ts:22`	Sensibilidad de cada nicho
`src/components/workspace/stages/OutlineReview.tsx:287`	Aviso en la pausa 1
`src/components/workspace/stages/TextEditor.tsx:151`	Bloque con candado en el editor
`src/lib/markdown.ts:24`	Posición en el libro exportado
`backend/app/guardrails.py:95`	Con backend: el mismo texto de aviso por categoría

Si te preguntan…

¿Por qué Gestión del tiempo no lleva aviso?
Porque es una habilidad práctica, no un consejo financiero, médico ni psicológico. Es una decisión registrada: si el equipo prefiere que todos los libros lo lleven, es un cambio de una línea en `niches.ts`.

¿Qué pasa si escribo el tema a mano?
Folio detecta el nicho por las palabras (por ejemplo, “ansiedad” → salud emocional) y aplica su aviso. Con el backend, además, el modelo de lenguaje clasifica la sensibilidad del tema, y si el nicho ya es sensible, nunca la baja.

¿Se puede quitar editando el archivo descargado?
Fuera de Folio, cualquiera puede editar un archivo. Lo que garantiza el sistema es que todo lo que exporta lo incluye.

REQUERIMIENTO ÉTICO Y DE RESPONSABILIDAD · 3 DE 3

RÉR-03

Mitigación de sesgos y estereotipos

Implementado; falta medirlo con el modelo real

QUÉ PIDE EL DOCUMENTO

Las situaciones cotidianas y ejemplos de la vida real generados no deben promover estereotipos de género, discriminación ni soluciones poco éticas a problemas económicos o sociales.

TU TURNO · DILO ASÍ

Los ejemplos usan doce personas, seis mujeres y seis hombres, con estudios y trabajos que no siguen estereotipos. Rotan a lo largo del libro y alternan el género.

Cómo lo cumple

- Perfiles pensados contra el estereotipo: Valentina estudia Ingeniería Mecánica y juega fútbol, José estudia Enfermería, Diego cuida a su hermano menor, Sofía repara celulares y Samuel acompaña a su abuela al médico.
- Las personas rotan por libro y dos capítulos seguidos nunca repiten género. En el libro de la prueba salieron Tomás, Valentina, Samuel, Daniela, Andrés, Sofía: 3 mujeres y 3 hombres.
- Las soluciones son hábitos responsables: planificar, ahorrar, conversar, pedir ayuda. Nada de atajos poco éticos.
- La misma regla se aplica a los libros de ejemplo y a los textos de ejemplo de la app.
- Con el backend, el prompt del Redactor exige personas diversas en nombre, edad, género, origen, familia y trabajo, roles que no dependen del género y soluciones éticas. Además, el orquestador le sugiere personas que rotan por el libro, como en el modo demo.

Cómo se comprueba

`RÉR-03 · Ejemplos con personas diversas`

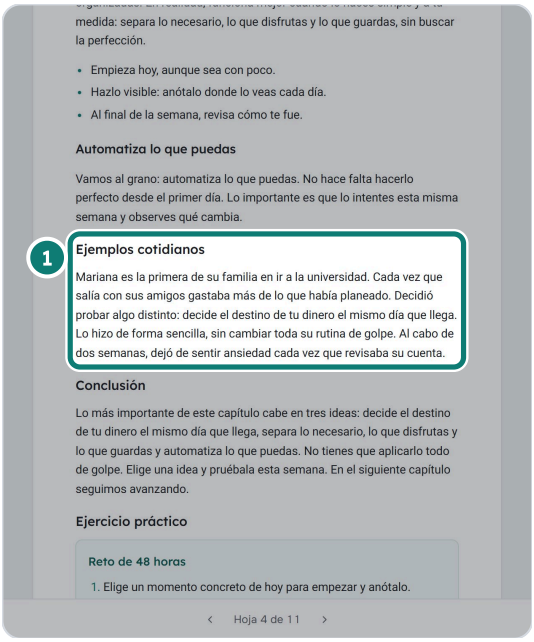
Extrae las personas de todos los ejemplos del libro y exige que aparezcan mujeres y hombres. En el backend, `test_personas_diversas_rer03` comprueba que las personas sugeridas rotan y tienen edades variadas.

Prueba superada · **3** mujeres · **3** hombres · Tomás · Valentina · Samuel · Daniela · Andrés · Sofía

Dónde verlo, paso a paso

1 Lee los ejemplos del capítulo 1

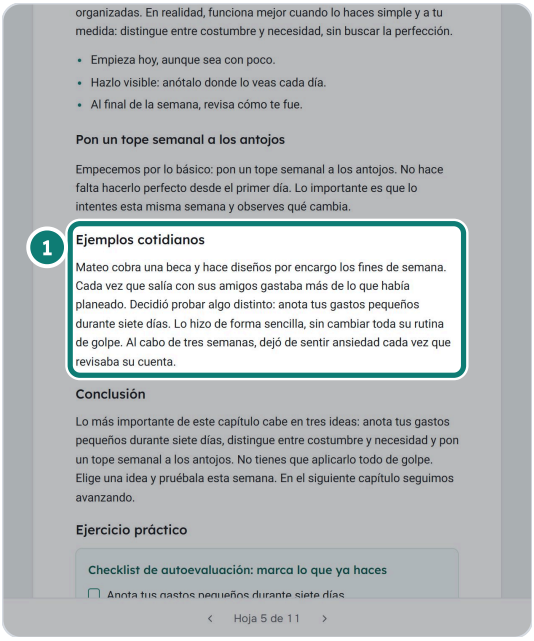
Abre “Ahorra sin dejar de vivir” y busca “Ejemplos cotidianos”.



1 Ejemplo del capítulo 1: Mariana, con un contexto concreto y una solución responsable.

2 Compáralos con el capítulo 2

Sigue bajando hasta los ejemplos del capítulo 2: cambian la persona y su género.



1 Ejemplo del capítulo 2: Mateo, con un contexto concreto y una solución responsable.

3 Repasa las doce personas

Estas son todas las personas de los ejemplos, con perfiles que evitan los estereotipos de género.

LAS DOCE PERSONAS DE LOS EJEMPLOS (CONTENT.TS)

Mujeres

- **Daniela** estudia de noche y trabaja en una cafetería
- **Valentina** estudia Ingeniería Mecánica y juega en el equipo de fútbol de la universidad
- **Camila** comparte piso con dos amigas y estudia Economía
- **Lucía** estudia a distancia desde un pueblo pequeño
- **Mariana** es la primera de su familia en ir a la universidad
- **Sofía** trabaja medio tiempo en un taller de reparación de celulares

Hombres

- **Mateo** cobra una beca y hace diseños por encargo los fines de semana
- **José** estudia Enfermería y hace entregas en bicicleta los fines de semana
- **Andrés** acaba de empezar su primer empleo en una oficina contable
- **Samuel** vive con su abuela y la acompaña a sus citas médicas
- **Tomás** volvió a estudiar después de unos años trabajando
- **Diego** estudia Derecho y cuida a su hermano menor por las tardes

Dónde está en el código

`src/lib/api/mock/content.ts:246`	Las doce personas y sus contextos
`src/lib/api/mock/content.ts:415`	Rotación por libro y alternancia de género
`backend/app/prompts/redactor/capitulo.md:33`	Con backend: reglas de diversidad del Redactor
`backend/app/agents/redactor.py:45`	Personas sugeridas que rotan por el libro

Qué falta medir

Revisar por muestreo los ejemplos de un libro escrito por Gemini y, si hace falta, una revisión automática de sesgos antes de mostrar cada capítulo.

Si te preguntan…

¿Doce personas no son pocas?

Para el modo demo alcanzan; lo importante es la regla. Con el backend, el Redactor crea sus propios ejemplos con las reglas de diversidad del prompt y parte de las personas sugeridas. Falta revisar por muestreo lo que escribe Gemini.

¿Qué encontró la verificación aquí?

Dos defectos: algunos perfiles repetían estereotipos y un libro salió con seis mujeres y ningún hombre. Se reescribieron los perfiles y ahora rotan alternando el género.

PARTE V

Funciones añadidas en la fase 4

Lo que el equipo sumó al construir el backend: una portada que puedes hacer tuya y ayuda con las referencias bibliográficas. No están en requerimientos.txt; se registraron como requerimientos derivados (D-37 y D-38).

RD-04	Edición de portada (ampliada)	73
RD-09	Ayuda con las referencias bibliográficas	79

REQUERIMIENTO DERIVADO · 1 DE 2

Cumple

RD-04

Edición de portada (ampliada)

QUÉ PIDE · DOCS/02, AMPLIADO EN D-37

Título, subtítulo, autor, plantilla de portada (cinco), color de tela, letra, tamaño y alineación del título, imagen propia (de fondo con un velo de tela que mantiene el contraste del título en 4,5:1 o más, o en un recuadro, con texto alternativo), contraportada con sinopsis que propone el Maquetador, letra y tamaño del cuerpo y tamaño de página, siempre dentro de las restricciones de RA-03.

TU TURNO · DILO ASÍ

La portada es tuya: eliges entre cinco plantillas, la letra, el tamaño y la alineación del título, y puedes subir una imagen propia. Si la pones de fondo, Folio calcula cuánto velo de color necesita para que el título se siga leyendo, y no te deja bajar de ahí. Y el libro cierra con una contraportada.

Cómo lo cumple

- "Portada y estilo" tiene tres vistas, cada una con su vista previa a la derecha: Portada, Contraportada e Interior.
- Cinco plantillas: Clásica, Centrada, Franja, Marco y Editorial. El título admite Lexend, Onest o Roboto, tres tamaños y alineación a la izquierda o centrada.
- Imagen propia en JPG, PNG o WebP de hasta 8 MB, de fondo o en recuadro, con encuadre y texto alternativo. Se reduce en el navegador antes de guardarse.
- De fondo lleva un velo del color de tela. Folio mide el 5 % de píxeles más claros de la imagen y calcula el velo mínimo para que el título blanco tenga 4,5:1 o más; con la imagen clara de la prueba salió de 84 %. El control no baja de ahí y, si cambias la tela, el mínimo se recalcula.
- Contraportada con la sinopsis que propone el Maquetador, editable y con "Pedir otra sinopsis". Es la última página del PDF y en el Markdown sale como "Sobre este libro".
- Con el backend, la API vuelve a validar la plantilla, la letra y el velo, y el PDF del servidor usa la misma portada, la imagen y la contraportada.

Cómo se comprueba

`RD-04 · Portada: plantillas, título, imagen con velo accesible y contraportada`

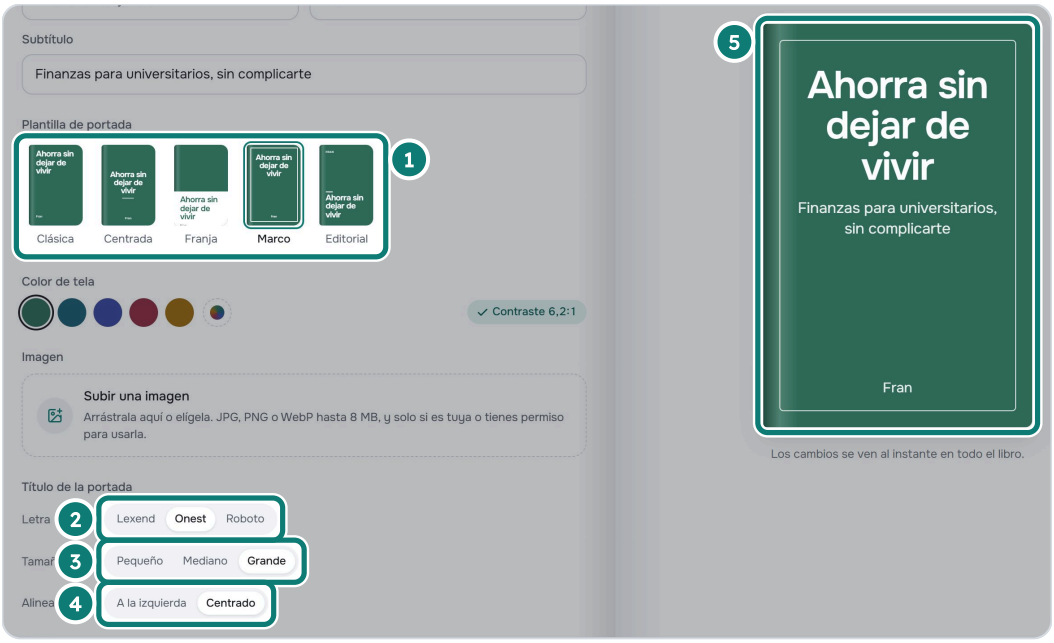
Elige la plantilla Marco, Onest y tamaño grande; sube una imagen clara y comprueba que el contraste con el velo es de 4,5:1 o más y que el control no baja del mínimo; luego la pasa a recuadro y pide otra sinopsis. En el backend, el servidor calcula el velo mínimo con el mismo algoritmo y lo aplica solo (`test_el_servidor_calcula_el_velo_minimo_y_lo_sube_en_silencio`).

Prueba superada · **5** plantillas · Velo mínimo **84 %** · Contraste **4,5:1** o más · Sinopsis **nueva**

Dónde verlo, paso a paso

1 Elige una plantilla y ajusta el título

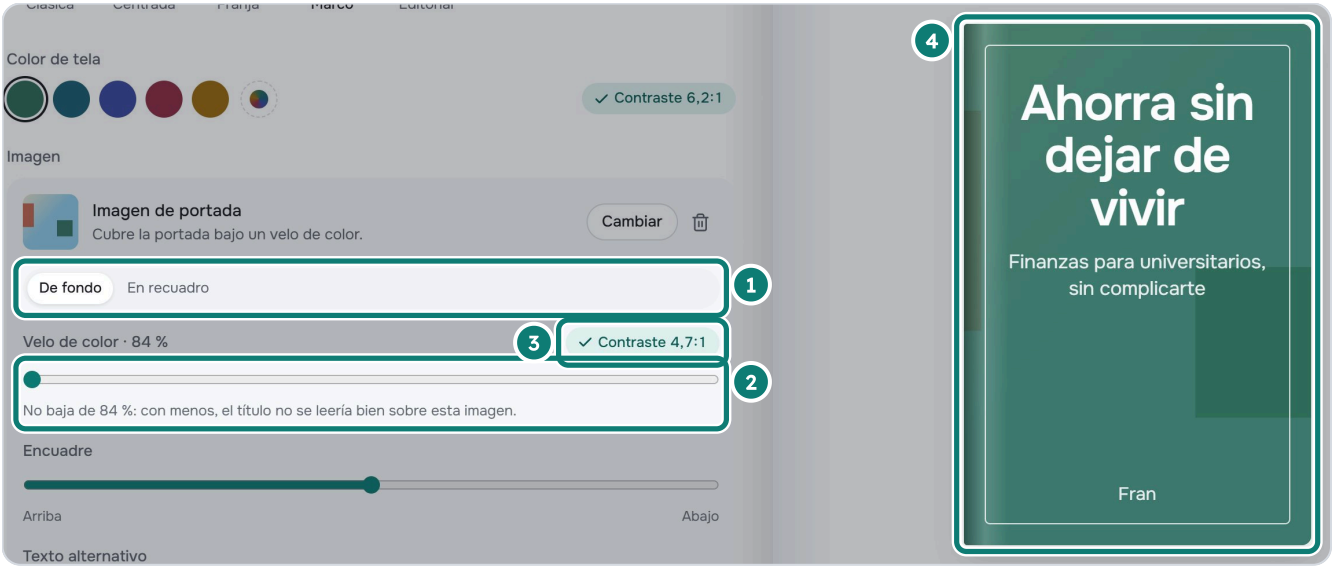
En “Ahorra sin dejar de vivir”, abre la pestaña “Portada y estilo”. En la vista “Portada”, toca la plantilla “Marco” y, más abajo, en “Título de la portada”, elige la letra Onest y el tamaño Grande.



1. Cinco plantillas: Clásica, Centrada, Franja, Marco y Editorial. Aquí está elegida Marco.
2. Letra del título: Lexend, Onest o Roboto.
3. Tamaño: pequeño, mediano o grande.
4. Alineación: a la izquierda o centrada. Cada plantilla propone una y puedes cambiarla.
5. La portada cambia al instante: Marco, Onest y tamaño grande.

2 Sube una imagen clara

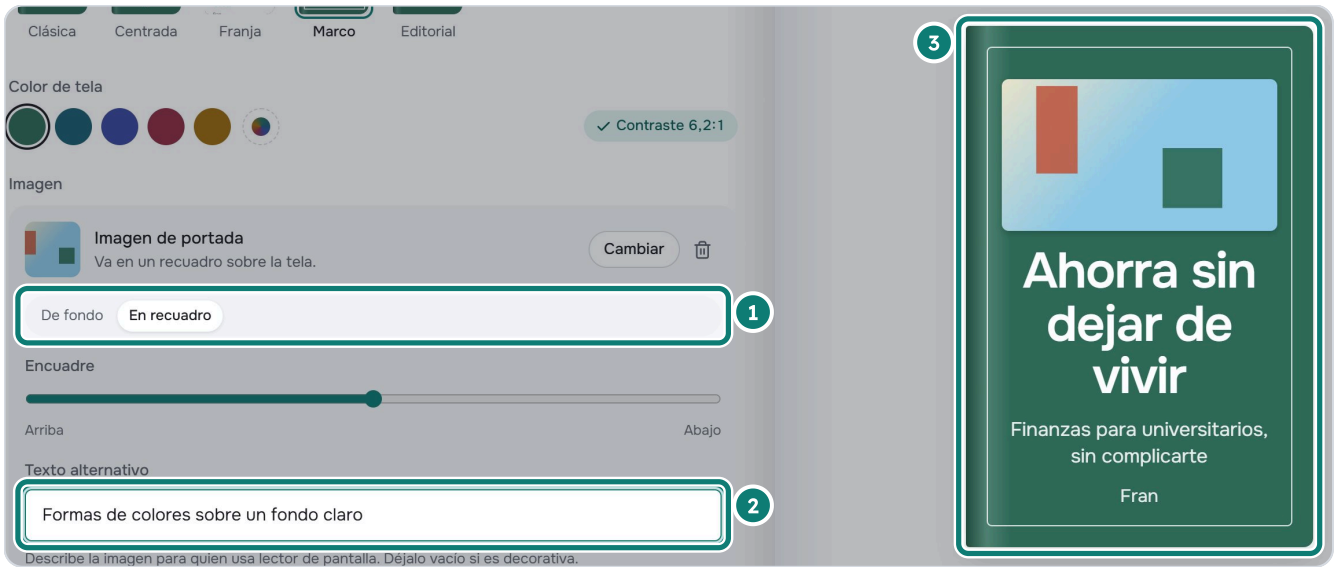
En “Imagen”, toca “Subir una imagen” y elige una foto clara. Queda de fondo, bajo un velo del color de tela. Prueba a mover el control del velo hacia la izquierda: se detiene en el mínimo.



1. La imagen queda de fondo. También puede ir en recuadro.
2. El velo: Folio lo calculó para esta imagen (84 %) y el control no deja bajar de ahí.
3. Con ese velo, el título blanco tiene un contraste de 4,7:1: cumple el mínimo de 4,5:1.
4. La portada con la imagen bajo el velo de la tela.

3 Pásala a un recuadro

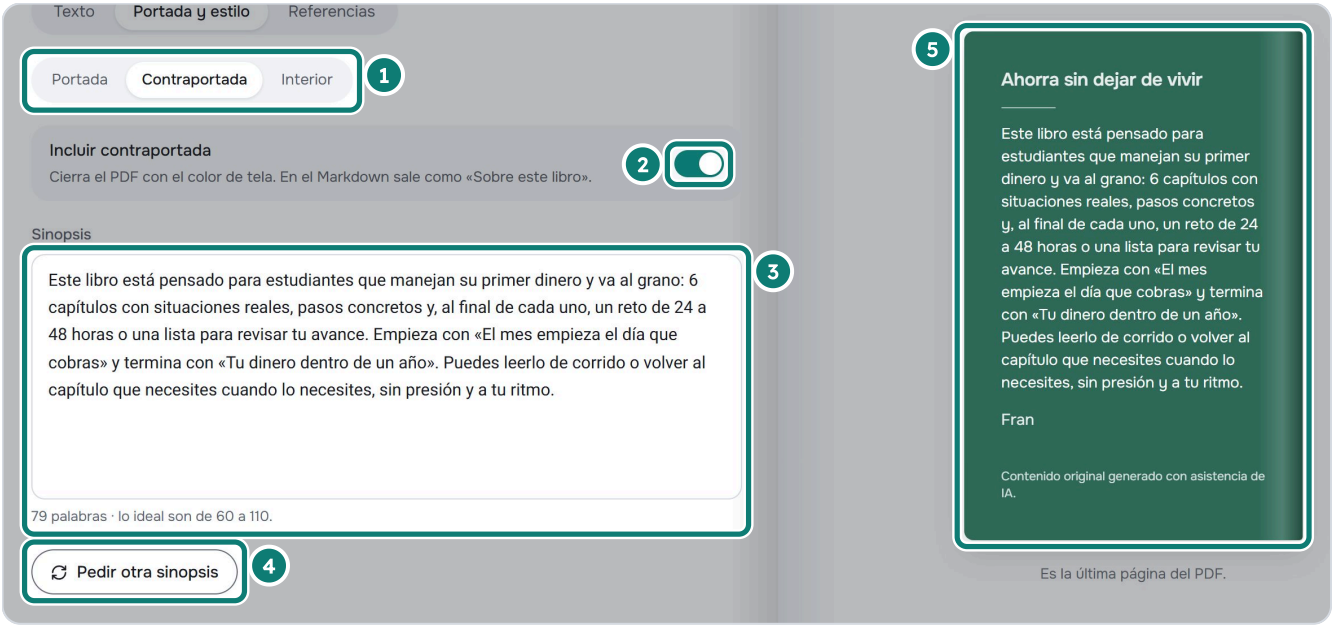
Elige “En recuadro” y escribe un texto alternativo que describa la imagen.



1. “En recuadro”: la imagen va sobre la tela, sin velo, y el título queda sobre el color liso.
2. El texto alternativo describe la imagen para quien usa lector de pantalla.
3. La portada con la imagen en recuadro.

4 Abre la contraportada

Cambia a la vista “Contraportada” y toca “Pedir otra sinopsis”.



1. Tres vistas, cada una con su vista previa a la derecha: Portada, Contraportada e Interior.
2. “Incluir contraportada”: cierra el PDF con el color de tela.
3. La sinopsis que propone el Maquetador: la puedes editar, y debajo ves cuántas palabras lleva.
4. “Pedir otra sinopsis” le pide una nueva al Maquetador.
5. A la derecha, la contraportada: es la última página del PDF.

Dónde está en el código

`src/components/workspace/stages/DesignPanel.tsx:62`	Las tres vistas: Portada, Contraportada e Interior
`src/lib/design.ts:12`	Las cinco plantillas y la alineación que propone cada una
`src/components/book/Cover.tsx:31`	La portada: plantilla, letra, tamaño, imagen y velo
`src/lib/cover-image.ts:64`	Color medio del 5 % de píxeles más claros
`src/lib/cover-image.ts:119`	Velo mínimo para llegar a 4,5:1
`src/components/workspace/stages/DesignPanel.tsx:435`	El control no baja del mínimo y explica por qué
`src/components/book/BackCover.tsx:10`	Contraportada
`backend/app/api/ebooks.py:659`	Con backend: el servidor calcula el velo mínimo y lo aplica
`backend/app/prompts/maquetador/contraportada.md:10`	Prompt de la sinopsis del Maquetador
`backend/app/export/templates/libro.html.j2:321`	Portada y contraportada en el PDF del servidor

Si te preguntan…

¿Por qué el velo no baja de cierto valor?

Porque el título va en blanco y tiene que leerse: WCAG pide 4,5:1 para texto normal, lo mismo que se exige a la tela sola (RA-03). Folio toma el color medio del 5 % de píxeles más claros, que es donde peor se leería el título, lo mezcla con la tela y busca la opacidad mínima que llega a 4,5:1, con un pequeño margen. Con una imagen oscura, el mínimo puede ser 0.

¿Y si quiero que se vea más la imagen?

Ponla “En recuadro”: va sobre la tela, sin velo, y el título queda sobre el color liso. Con una imagen muy clara de fondo, el velo mínimo es alto a propósito.

¿Puedo usar cualquier imagen?

Solo si es tuya o tienes permiso, y lo dice la propia pantalla. Folio no genera imágenes con IA (D-15) y pide un texto alternativo para quien usa lector de pantalla.

¿Por qué el backend vuelve a validar el velo?

Porque la interfaz se puede saltar llamando a la API. El servidor no se fía del cliente: calcula el velo mínimo a partir de la imagen y de la tela con el mismo algoritmo, y si llega uno menor lo sube solo.

¿Quién escribe la sinopsis?

El Agente Maquetador propone una de 60 a 110 palabras al terminar el libro. La puedes editar o pedir otra.

REQUERIMIENTO DERIVADO · 2 DE 2

Cumple

RD-09

Ayuda con las referencias bibliográficas

QUÉ PIDE · DOCS/02, AÑADIDO EN D-38

El Investigador sugiere lecturas reales relacionadas con el tema; Folio verifica que existan en Crossref (artículos, DOI) y Open Library (libros, ISBN) y marca las que no encuentra; el usuario busca por título, DOI o ISBN, agrega a mano, edita, corrige con los datos de la fuente y copia la cita en el texto. Se exportan al final del libro en APA 7 (versión en español). Apoya RÉR-01 (originalidad y transparencia de fuentes).

TU TURNO · DILO ASÍ

Al aprobar el índice, el Investigador sugiere lecturas reales y Folio comprueba en Crossref y Open Library que existan. Si alguna no aparece, te lo dice claro antes de exportar. Puedes buscar por DOI o ISBN, agregar las tuyas y copiar la cita en APA 7.

Cómo lo cumple

- Al aprobar el índice, el Investigador sugiere lecturas reales relacionadas con el tema y los capítulos, en una tarea aparte que no frena la redacción. Su prompt le prohíbe inventar obras, DOI o ISBN.
- Cada referencia se verifica: con DOI en Crossref, con ISBN en Open Library y, si no, por título y autor. Cuenta como encontrada si el título se parece en un 80 % o más y coincide algún apellido.
- Hay tres resultados: “Verificada” (con la fuente), “No la encontramos” (posible invención) y “Sin verificar” (sin conexión o página web). Si la fuente tiene otro año o datos que faltan, ofrece “Usar esos datos”.
- En la pestaña Referencias buscas por título, autor, DOI o ISBN, agregas a mano (libro, artículo o página web), editas, quitas, vuelves a comprobar y copias la cita, por ejemplo “(Housel, 2020)”.
- Se exportan al final del libro, en la sección Referencias, en APA 7 en español y ordenadas por apellido. El diálogo de exportar avisa si alguna no se encontró.
- Con el backend, el servidor hace la misma búsqueda y verificación, y aplica las mismas reglas de APA 7 que el frontend.

Cómo se comprueba

`RD-09 · Referencias sugeridas, verificadas, editables y en APA 7`

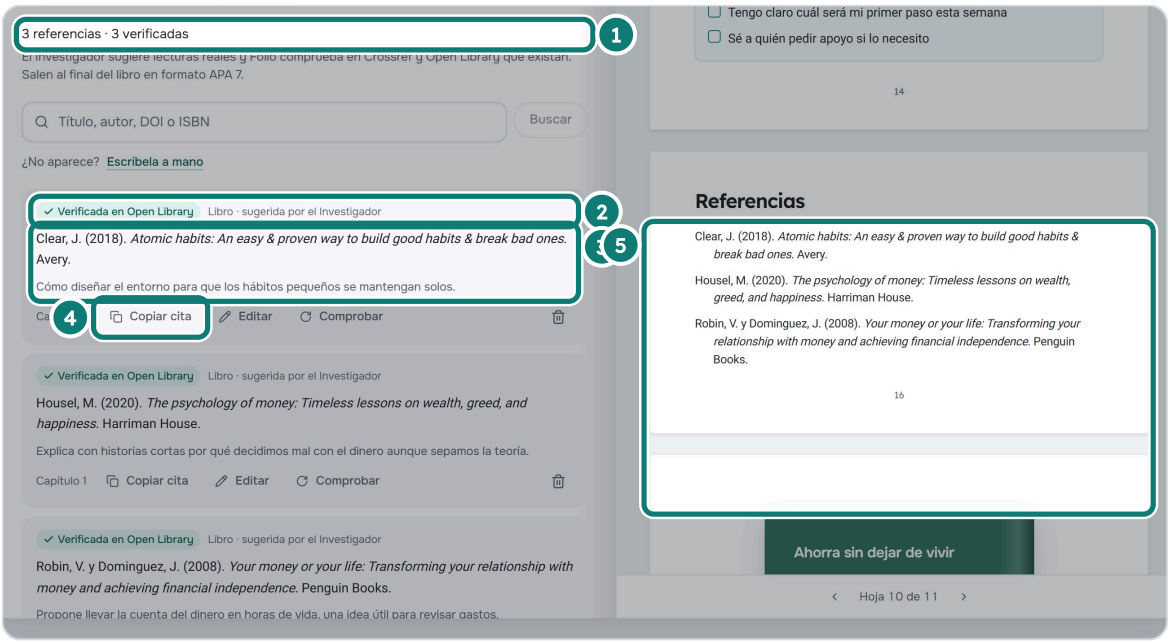
Con Crossref y Open Library simuladas (sin red), comprueba las lecturas verificadas del libro de ejemplo; agrega una por DOI, que queda “Verificada en Crossref”, y otra inventada, que queda “No la encontramos”; copia la cita “(Housel, 2020)” y exporta el Markdown con la sección Referencias en APA 7. En el backend hay 20 pruebas más entre `test_references.py` y `test_apa.py`.

Prueba superada · **3** sugeridas por el Investigador · **4** de 5 verificadas · **1** inventada, detectada · Cita **(Housel, 2020)**

Dónde verlo, paso a paso

1 Abre la pestaña Referencias

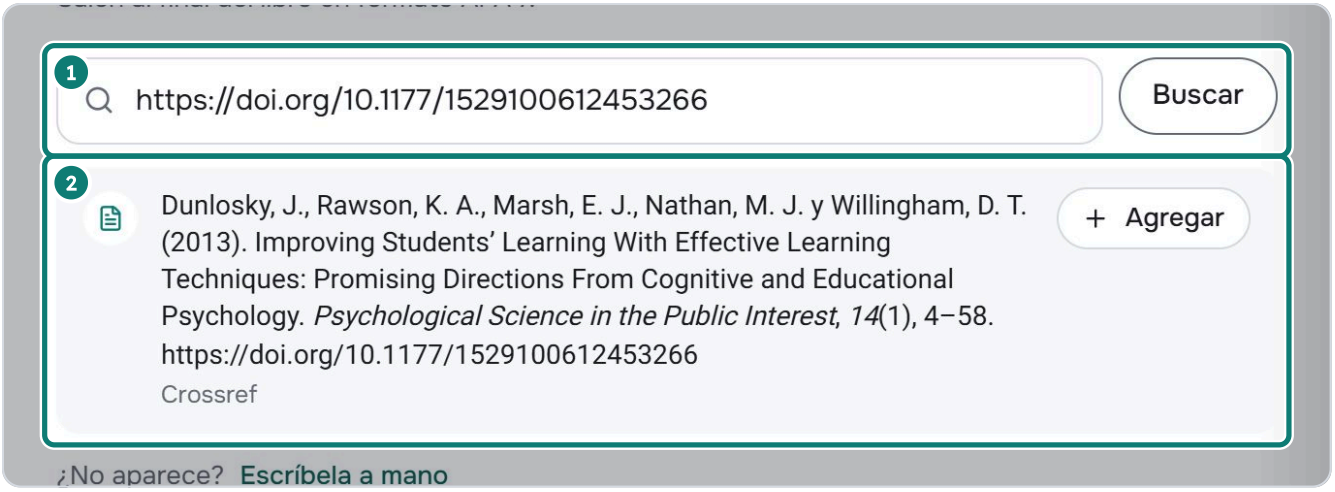
En “Ahorra sin dejar de vivir”, toca “Referencias”. Ahí están las lecturas que sugirió el Investigador.



1. El resumen: cuántas referencias hay y cuántas se verificaron.
2. Cada una dice si se verificó, dónde, y si la sugirió el Investigador o la agregaste tú.
3. La referencia ya en APA 7, con el motivo por el que te sirve.
4. “Copiar cita” copia “(Clear, 2018)” para pegarla donde uses la idea.
5. En “Tu libro”, las referencias van al final, en su propia página.

2 Busca una obra por DOI

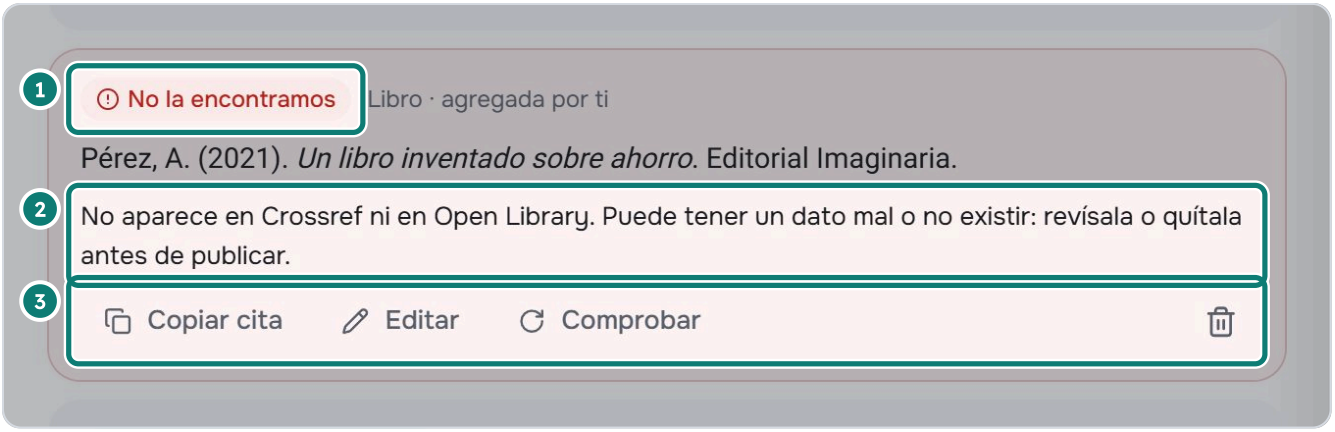
En el buscador, pega `https://doi.org/10.1177/1529100612453266` y toca “Buscar”. Después, toca “Agregar”.



1. Pegas el DOI, o el enlace de doi.org, y tocas “Buscar”.
2. Crossref devuelve la obra y Folio la muestra ya en APA 7. “Agregar” la suma a tu lista.

3 Agrega una obra que no existe

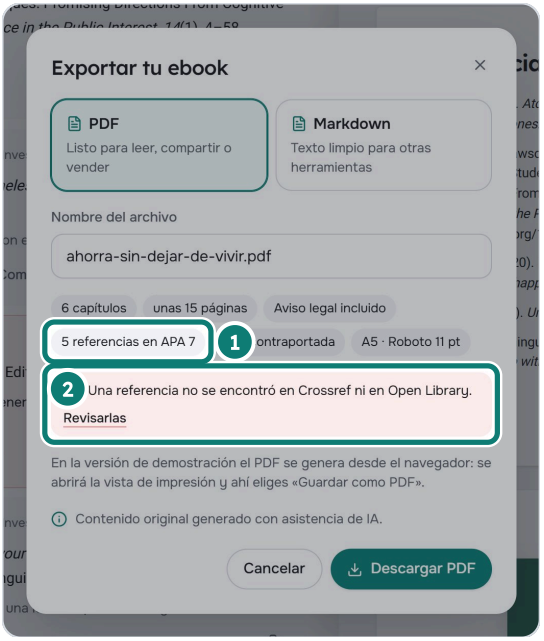
Toca “Escríbela a mano” y completa una obra inventada: autora “Pérez, Ana”, título “Un libro inventado sobre ahorro”, año 2021. Guárdala.



1. No aparece en Crossref ni en Open Library: queda como “No la encontramos”.
2. Folio dice qué hacer: revisarla o quitarla antes de publicar.
3. Como cualquier otra, puedes copiar la cita, editarla, volver a comprobarla o quitarla.

4 Abre Exportar

Toca “Exportar” en la cabecera.



1. El resumen cuenta las referencias que irán al final del libro, en APA 7.
2. Si una no se encontró, lo avisa antes de descargar, y “Revisarlas” te lleva a la pestaña Referencias.

5 Mira el Markdown

Descarga el Markdown y busca la sección Referencias, después del último capítulo:

ahorra-sin-dejar-de-vivir.md

```
295 # Referencias                                                          1
296
297 Clear, J. (2018). *Atomic habits: An easy & proven way to build good habits & break bad ones*.  2
    Avery.
298
299 Housel, M. (2020). *The psychology of money: Timeless lessons on wealth, greed, and happiness*.
    Harriman House.
300
301 Robin, V. y Dominguez, J. (2008). *Your money or your life: Transforming your relationship with  3
    money and achieving financial independence*. Penguin Books.
```

1. Las referencias son un H1, después del último capítulo.
2. Cada una en APA 7: autor con iniciales, año, título en cursiva y editorial.
3. Versión en español: “y” entre autores. El orden es alfabético por apellido.

Dónde está en el código

`src/components/workspace/stages/ReferencesPanel.tsx:35`	Pestaña Referencias
`src/components/workspace/stages/ReferencesPanel.tsx:237`	Verificada, No la encontramos o Sin verificar
`src/lib/references/lookup.ts:291`	Verificación en Crossref y Open Library (modo demo)
`src/lib/references/lookup.ts:238`	Umbral de parecido del título: 80 %
`src/lib/references/apa.ts:88`	Formato APA 7 en español
`src/lib/references/apa.ts:155`	Cita en el texto: (Autor, año)
`src/components/workspace/ExportDialog.tsx:131`	Aviso al exportar
`backend/app/prompts/investigador/lecturas.md:10`	Con backend: prompt de lecturas del Investigador
`backend/app/references/verify.py:163`	La misma verificación en el servidor
`backend/app/references/apa.py:138`	El mismo formato APA 7 en el servidor

Si te preguntan…

¿Qué pasa si la IA inventa una referencia?
Es un riesgo conocido de los modelos de lenguaje. Por eso el prompt del Investigador le prohíbe inventar y le pide dejar vacío cualquier dato dudoso, y Folio comprueba cada obra en Crossref y Open Library. Si no aparece, queda como “No la encontramos”, la pestaña muestra cuántas hay por revisar y el diálogo de exportar lo advierte.

¿Por qué no se citan solas en el texto?
Porque el modelo podría atribuir a una obra una idea que la obra no dice. Folio no inserta citas automáticas: te da la cita “(Autor, año)” para que la pegues donde de verdad uses la idea (D-38).

¿Y si no hay conexión?
La referencia queda “Sin verificar” y puedes volver a comprobarla después. Las páginas web no se verifican: abre el enlace para confirmar que sigue disponible.

¿Por qué Crossref y Open Library?
Son abiertas, gratuitas y se pueden consultar desde el navegador. Crossref tiene los DOI de artículos y libros académicos; Open Library, los ISBN y los datos de millones de libros.

¿Las pruebas dependen de esas bases?
No: en las pruebas se simulan, así que el resultado no cambia si Crossref u Open Library fallan. Contra los servicios reales se comprobaron a mano las lecturas sugeridas al construir el backend.

PARA LA PRESENTACIÓN

Guion de demostración en 5 minutos

Un solo recorrido por el sitio publicado (modo demo) que toca los 16 requerimientos y las dos funciones añadidas. Antes de empezar, pon la velocidad en Rápida, ten a mano una imagen clara (JPG o PNG) y los libros de ejemplo por si el tiempo aprieta.

0:00 **Inicio**

Muestra los chips de nichos y escribe “Escribe una novela de dragones”.

> “Folio escribe guías prácticas de crecimiento personal sobre situaciones reales. Si le pido ficción, no crea el libro y me propone alternativas.”

RF-01

0:30 **Inicio**

Toca “Finanzas para universitarios”, elige 5 capítulos y envía.

> “Elijo el tema y el número de capítulos, que solo puede ir de 5 a 7. Ahora trabaja el Investigador, uno de los cuatro agentes.”

RF-02 RNF-04

1:00 **Revisión del índice**

Señala la cinta, las papeleras desactivadas y el aviso legal. Agrega capítulos hasta 7 y aprueba.

> “Primera pausa: el sistema se detiene y me espera. Con 5 no puedo quitar y con 7 no puedo agregar. Como el tema es financiero, ya me avisa que llevará aviso legal.”

RF-05 RF-02 RÉR-02

1:40 **Capítulo 1**

Mira “Tu libro” mientras se escribe.

> “El texto aparece en vivo, sección por sección. Cada capítulo tiene las mismas cinco partes.”

RNF-02 RF-03

2:05 **Revisión del capítulo 1**

Toca “Más cercano” → “Aplicar ajustes”. Muestra el ejercicio del final y aprueba.

> “Segunda pausa: este capítulo marca el tono de todo el libro y lo puedo ajustar con un toque. El ejercicio solo puede ser un checklist o un reto de 24 a 48 horas.”

RF-05 RNF-01 RNF-03 RF-04

2:45 **Escribiendo tu libro**

Señala que solo un capítulo dice “Escribiendo”.

> “Los capítulos se escriben en orden, siguiendo al capítulo 1, para que el libro sea coherente. Los ejemplos usan personas diversas, sin estereotipos.”

RNF-03 RÉR-03

3:05 **Libro final**

En Texto, elige “Aviso legal” (bloqueado). En Portada y estilo, pon un amarillo claro y usa el tono accesible.

> “El aviso legal no se puede quitar. El PDF sale con hoja blanca, Roboto, Arial o Helvetica desde 11 pt, y la portada siempre tiene buen contraste.”

RÉR-02 RA-03 RA-01

3:35 **Portada y referencias**

Sube una imagen clara de fondo y mueve el velo a la izquierda. Después, abre Referencias y muestra las verificadas.

> “La portada es mía, pero el velo no baja de lo que necesita el título para leerse. El Investigador sugiere lecturas reales y Folio comprueba que existan antes de citarlas.”

RD-04 RD-09

4:10 **Exportar**

Descarga el Markdown y abre la vista de impresión del PDF.

> “Exporto en Markdown y PDF, con encabezados H1, H2 y H3 para lectores de pantalla, las referencias en APA 7 y la nota de contenido original.”

RF-06 RA-01 RÉR-01

4:35 Cierre

Vuelve a la biblioteca.

> “El texto es claro: INFLESZ 78,8. Todo esto funciona también con el backend: cuatro agentes en Python con LangGraph, Gemini detrás de una capa intercambiable y el PDF hecho en el servidor. Lo que falta es medir la calidad del texto con Gemini y publicar el backend.”

RA-02 RNF-04

SI EL TIEMPO APRIETA

Después del paso 3, salta a “Hablar claro sin pelear”, que ya está en la segunda pausa, y luego a “Ahorra sin dejar de vivir”, que ya está listo para exportar.

SI QUIERES MOSTRARLO CON EL BACKEND

Arráncalo con `.\iniciar-folio.ps1` antes de la presentación y crea una cuenta. El guion es el mismo, pero no hay libros de ejemplo y cada capítulo tarda lo que tarde el modelo, así que conviene llevar un libro ya listo. Sin clave de Gemini en `backend/.env`, el backend usa el proveedor demo.

PARA LA RONDA DE PREGUNTAS

Preguntas frecuentes

Preguntas generales sobre el proyecto que no son de un requerimiento en particular.

¿El sitio publicado usa IA real?

No. GitHub Pages solo sirve archivos estáticos, así que publica el modo demo: los cuatro agentes se simulan en el navegador con textos de plantilla. Los agentes reales viven en el backend (FastAPI y LangGraph), que hoy se ejecuta en local. Los dos modos usan la misma fachada `api`, así que la interfaz es la misma.

¿Qué son los datos simulados (mocks)?

Código que imita al backend dentro del navegador: genera índices y capítulos de plantilla, con streaming y pausas, y guarda todo en el almacenamiento local. Sirvió para construir y probar la interfaz antes del backend, y hoy es el modo demo.

¿Dónde se guardan los libros?

En el modo demo, en el navegador (localStorage), separados por usuario. Con el backend, en una base SQLite local, con cuentas propias (contraseña protegida con scrypt y sesión con JWT). Supabase queda como opción si se despliega (D-35).

¿Cómo sé que las pruebas son de verdad?

Hay tres niveles. 20 pruebas de Playwright abren Edge y usan la app como lo haría una persona, en el modo demo. 147 pruebas de pytest recorren el backend por su API, sin salir a la red. Y una prueba punta a punta levanta el backend real y el frontend conectado y hace el recorrido completo. Todas fallan si algo no se cumple, y encontraron siete defectos reales que se corrigieron.

¿Por qué seis requerimientos no están en “Cumple”?

Porque hablan de cómo escribe la IA. El backend tiene los mecanismos, pero en la verificación no había clave de Gemini y las pruebas usaron el proveedor demo, que devuelve plantillas. Marcarlos como cumplidos sin leer textos reales del modelo no sería honesto.

¿Qué pasa si recargo en medio de la redacción?

El trabajo se retoma donde iba y las pausas se conservan (RF-05). Con el backend, incluso si se reinicia el servidor: cada capítulo es un checkpoint guardado en SQLite.

¿Con qué está hecho?

Frontend: Next.js 16, React 19, TypeScript, Tailwind CSS 4, Radix UI y Motion. Backend: Python con FastAPI y LangGraph, streaming por SSE, SQLite y Chromium para el PDF. El modelo por defecto es Gemini, detrás de una capa que también admite Claude, OpenAI o un proveedor demo sin conexión (D-34).

¿Cómo arranco Folio con los agentes reales?

En Windows, `.\iniciar-folio.ps1` desde la raíz del repositorio instala lo que falte y arranca el backend y el frontend conectados. Pon la clave de Gemini en `backend/.env`; sin ella se usa el proveedor demo. Los detalles están en `backend/README.md`.

¿Cómo se publica el sitio?

El frontend, con GitHub Pages: cada push a main que toque el frontend compila la versión estática (el modo demo) y la publica sola. El backend todavía no está desplegado: hace falta un servidor con Python y Chromium (P-08).

CALIDAD

Lo que encontró la verificación

Las pruebas no solo confirmaron: encontraron siete defectos reales, cuatro en la fase 3 y tres al construir el backend, y todos se corrigieron. Es un buen ejemplo para explicar por qué cada requerimiento tiene su prueba.

REQ.	QUÉ PASABA	CÓMO SE CORRIGIÓ
RÉR-03	Algunos perfiles repetían estereotipos (mujer enfermera y cuidadora, hombre futbolista).	Se reescribieron con roles no estereotipados.
RÉR-03	Las personas se elegían al azar por capítulo y un libro salió con seis mujeres y ningún hombre: la prueba falló.	Ahora rotan a lo largo del libro y alternan el género.
RA-01	El PDF no tenía H1: el título estaba oculto y los niveles empezaban en H2.	Portadilla visible con el título en H1; capítulos en H1, secciones en H2 y subtemas en H3.
RA-01	En la vista previa, los subtemas tenían el mismo nivel que las secciones.	Los subtemas bajaron un nivel.
RD-03	En el libro abierto, ninguna de las dos páginas tenía scroll propio: la vista previa nunca se desplazaba al capítulo que se editaba.	La fila del libro abierto se limita a la altura de la pantalla y cada página se desplaza por separado.
RA-01	En el PDF del servidor, los marcadores de los títulos largos perdían espacios (“estudia,descansa”): Chromium junta las líneas sin espacio.	Se restaura el texto real de cada marcador tras generar el PDF; lo prueba `test_pdf_outline.py`.
RD-06	Las miniaturas de plantilla leían todo el texto de la portada como nombre del botón.	La miniatura es decorativa y el botón se llama como la plantilla.

ESTADO DEL PROYECTO

Qué trajo la fase 4 y qué falta

El backend ya está construido y la interfaz habla con él sin cambiar sus componentes. Lo que queda no es código nuevo, sino medir y publicar: la calidad del texto con Gemini, el PDF con un lector de pantalla y el despliegue. Cómo está hecho el backend: `docs/11-backend.md`; cómo arrancarlo: `backend/README.md`.

Lo que ya funciona con el backend

REQ.	QUÉ HAY
RNF-04	Backend con FastAPI y LangGraph: los cuatro agentes como módulos, un archivo de prompt por tarea y el modelo detrás de una capa intercambiable (Gemini por defecto).
RF-05	Pausas con `interrupt()` y checkpoints en SQLite: si el servidor se reinicia, cada libro sigue donde estaba.
RF-01 RÉR-02	La ficción y los temas sensibles también los clasifica el modelo, en el servidor.
RF-02 RF-03 RF-04	La API y los agentes vuelven a validar el rango de capítulos, las cuatro secciones y el tipo de ejercicio.
RF-06 RA-01 RA-03	Markdown y PDF etiquetado generados por el Agente Maquetador en el servidor, con las fuentes incrustadas y la marca de IA.
RNF-01 RNF-02 RNF-03 RA-02 RÉR-01 RÉR-03	Los mecanismos para escribir bien: reglas en los prompts, capítulo 1 como ancla, guía de estilo, tiempo límite de 60 s y el filtro de recitación.
RD-04 RD-09	Dos funciones nuevas: la portada ampliada con contraportada y las referencias verificadas en APA 7.

Lo que falta

REQ.	QUÉ FALTA
RNF-01	Evaluar el tono de capítulos escritos por Gemini con la rúbrica de docs/02 §2.
RNF-02	Medir el tiempo real por capítulo con Gemini (meta: el 95 % por debajo de 60 s).
RNF-03	Leer un libro completo escrito por Gemini y buscar repeticiones o contradicciones.

REQ.	QUÉ FALTA
RA-02	Calcular INFLESZ sobre ese mismo libro (mínimo 55).
RÉR-01 RÉR-03	Revisar por muestreo la originalidad y los ejemplos y, si hace falta, añadir una verificación de similitud y una revisión automática de sesgos.
RA-01	Revisar el PDF del servidor con un lector de pantalla real (NVDA).
P-08	Desplegar el backend: hace falta un servidor con Python y Chromium. Hoy corre en local y GitHub Pages publica el modo demo.

REFERENCIA

Glosario

Agente
Un programa con un rol concreto (investigar, redactar, crear ejercicios o maquetar) que usa un modelo de lenguaje con sus propias instrucciones.

APA 7
Norma de citas de la American Psychological Association, 7.ª edición. Folio la usa en su versión en español.

Aviso legal
Texto que aclara que el libro es educativo y no sustituye a un profesional. Obligatorio en temas sensibles (RÉR-02).

Checkpointer
La pieza de LangGraph que guarda el estado del grafo (en Folio, en SQLite) para retomarlo después de una pausa o de un reinicio.

Contraste
Relación de luminancia entre el texto y el fondo. WCAG pide al menos 4,5:1 para texto normal.

Crossref
Registro abierto de los DOI de artículos y libros académicos. Folio lo consulta para verificar referencias.

DOI
Identificador permanente de un artículo o libro académico, por ejemplo 10.1177/1529100612453266.

Fachada api
El único punto por el que la interfaz pide datos. Según el modo, responde el simulador del navegador (demo) o el backend (REST y SSE).

FastAPI
Framework de Python para construir APIs web. En Folio sirve las rutas REST y el canal SSE.

Gemini
El modelo de lenguaje de Google que el backend usa por defecto (D-34).

Guardarraíl
Una regla que filtra la entrada antes de que trabajen los agentes; aquí, rechazar la ficción.

HITL
Human in the loop: una persona aprueba los pasos críticos de un proceso automático.

INFLESZ
Escala de legibilidad para textos en español. De 65 a 80 es “bastante fácil”.

LangGraph
Librería de Python para construir flujos de agentes como un grafo, con pausas y estado guardado.

Lector de pantalla
Programa que lee en voz alta lo que hay en pantalla: NVDA, VoiceOver o TalkBack.

localStorage
Almacenamiento del navegador donde el modo demo guarda los libros de cada usuario.

Markdown
Formato de texto con marcas simples: # para títulos, - para listas y > para citas.

Mock
Código que imita al backend para construir y probar la interfaz sin él. En Folio es el modo demo.

Modo demo
La versión publicada: los agentes se simulan en el navegador con textos de plantilla.

Nicho
El área temática del libro: finanzas, tiempo, conflictos, primer empleo…

Open Library
Catálogo abierto de libros del Internet Archive. Folio busca ahí por ISBN o por título y autor.

PDF etiquetado
PDF que guarda la estructura del documento (títulos, párrafos, orden de lectura) además de su aspecto.

Playwright
Herramienta que controla un navegador para automatizar pruebas como si fuera una persona.

Prompt
Las instrucciones que recibe el modelo de lenguaje. En Folio, cada tarea de cada agente tiene su archivo.

Proveedor demo
Proveedor del backend que funciona sin conexión y devuelve textos de plantilla. Sirve para probar el flujo, no la calidad.

Punto de control
Cada una de las dos pausas en las que apruebas: el índice y el capítulo 1.

SQLite
Base de datos en un solo archivo. Guarda las cuentas, los ebooks, las imágenes y los checkpoints.

SSE
Server-Sent Events: canal por el que el servidor envía eventos al navegador en tiempo real.

Streaming
Mostrar el texto a medida que se genera, en lugar de esperar a que termine.

Temperatura
Parámetro del modelo que regula cuánto varía el texto: baja es predecible y alta, más creativa.

Velo
Capa del color de tela sobre la imagen de portada. Su opacidad mínima asegura 4,5:1 para el título.

WCAG 2.2 AA
Pautas internacionales de accesibilidad web. AA es el nivel que se exige habitualmente.

REFERENCIA

Cómo repetir las pruebas

Cualquiera del equipo puede volver a comprobarlo todo. Para el frontend hace falta Node.js 20 o superior y Microsoft Edge; para el backend, Python 3.12 o 3.13.

Frontend, en el modo demo

```
cd frontend
npm install
npm run test:e2e
```

Son 20 pruebas y tardan unos dos minutos. Guardan las capturas en `frontend/e2e/evidencias/` y un informe HTML en `frontend/e2e/informe/`.

Backend

```
cd backend
.venv/Scripts/python -m pytest      # 147 pruebas, unos 40 s, sin salir a la red

cd ../frontend
npm run test:e2e:backend            # punta a punta con el backend real
```

La punta a punta usa el proveedor demo y datos temporales. Cómo crear el entorno de Python está en `backend/README.md`.

Contra el sitio publicado

```
# Git Bash, macOS o Linux
E2E_BASE_URL=https://franjmd0508.github.io/multiagente-inteligente-ebooks npm run test:e2e

# PowerShell
$env:E2E_BASE_URL="https://franjmd0508.github.io/multiagente-inteligente-ebooks"; npm run
test:e2e
```

Prueba lo que esté publicado en ese momento: las pruebas de RD-04 y RD-09 solo pasan cuando el sitio ya tiene la versión de la fase 4.

Volver a generar esta guía

```
npm run guia:capturas   # recorre la app y toma las capturas señaladas (~2 min)
npm run guia:pdf        # arma el PDF (~30 s)
```

Las capturas quedan en `docs/guia/capturas/`, cada una con un JSON que dice dónde van las marcas.

Dónde está cada cosa

`docs/09-verificacion-requerimientos.md`	Esta verificación en texto, con las mismas evidencias.
`docs/verificacion/`	Capturas, datos medidos y los ejemplos exportados (.md y .pdf).
`frontend/e2e/requisitos.spec.ts`	Las 18 pruebas de aceptación de los 16 requerimientos.
`frontend/e2e/portada-referencias.spec.ts`	Las 2 pruebas de las funciones añadidas (RD-04 y RD-09).
`frontend/e2e/backend.spec.ts`	El recorrido punta a punta con el backend real.
`backend/tests/`	Las 147 pruebas del backend (pytest).
`docs/11-backend.md`	Cómo está hecho el backend: grafo, agentes, prompts y pruebas.
`frontend/e2e/capturas-guia.spec.ts`	El recorrido que toma las capturas de esta guía.
`frontend/scripts/guia-requerimientos.mts`	El generador de esta guía.

Folio

Ebooks cortos de crecimiento personal, escritos por cuatro agentes y aprobados por ti.

franjmd0508.github.io/multiagente-inteligente-ebooks/

github.com/FranJMD0508/multiagente-inteligente-ebooks

Equipo Folio · Ingeniería de Software · UJAP · 2026